

JÚLIA CARVALHO MUNIZ
KAUÃ FILLIPE RODRIGUES DA SILVA

**MODERNIZAÇÃO TECNOLÓGICA DE SIMULADOR DE REALIDADE
VIRTUAL PARA TREINAMENTO DO PROCEDIMENTO DE PUNÇÃO
VENOSA COM APLICAÇÃO EM EDUCAÇÃO DE ENFERMAGEM**

São Paulo
2025

JÚLIA CARVALHO MUNIZ
KAUÃ FILLIPE RODRIGUES DA SILVA

**MODERNIZAÇÃO TECNOLÓGICA DE SIMULADOR DE REALIDADE
VIRTUAL PARA TREINAMENTO DO PROCEDIMENTO DE PUNÇÃO
VENOSA COM APLICAÇÃO EM EDUCAÇÃO DE ENFERMAGEM**

Dissertação apresentada ao Departamento de Engenharia de Computação e Sistemas Digitais da Escola Politécnica da Universidade de São Paulo para obtenção do Título de Engenheiro.

Área de Concentração: Engenharia da Computação

Orientador: Prof. Dr. Ricardo Nakamura
Coorientadora: Profa. Dra. Simone de Godoy Costa

São Paulo
2025

AGRADECIMENTOS

Ao Prof. Dr. Ricardo Nakamura, por toda a orientação, dedicação e apoio ao longo do desenvolvimento deste trabalho. Sua *expertise* técnica e paciência diante dos inúmeros desafios enfrentados durante o projeto foram fundamentais para o êxito deste trabalho. Agradecemos profundamente por ter acreditado no potencial deste projeto desde o início e por ter estado presente em cada etapa, oferecendo não apenas orientação acadêmica, mas também inspiração e motivação para perseverar mesmo nos momentos mais desafiadores.

À Profa. Dra. Simone de Godoy Costa, por toda disposição, generosidade e paciência em nos apresentar o legado do projeto VIDA Enfermagem. Sua dedicação em nos transmitir os conceitos essenciais da área de Enfermagem e sua preocupação genuína em garantir que o projeto fosse retomado com o devido respeito e qualidade nos motivaram profundamente.

Aos nossos pais, por todo o apoio incondicional à nossa educação e pela compreensão infinita durante as inúmeras noites de dedicação a este trabalho. Vocês são a base de tudo o que conquistamos e o alicerce que nos permitiu chegar até aqui. Esta conquista é tanto nossa quanto de vocês.

Aos nossos amigos, por terem estado junto conosco não só nos picos, mas principalmente nos vales desta graduação. Vocês tornaram esta jornada mais leve e significativa.

A Deus, por fim, por sua presença constante, mesmo nas horas de maior dificuldade. Por estar sempre mais perto do que imaginamos – mais próximo que nossa própria veia jugular.

A todos vocês, nossos sinceros e profundos agradecimentos.

“Meu caminho é novo, mas meu povo não.”

Anavitória

RESUMO

A *Punção Venosa Periférica* (PVP) é um procedimento amplamente realizado na rotina de enfermeiros, mas estudantes frequentemente apresentam dificuldades em sua execução devido à limitada oportunidade de prática supervisionada. Este trabalho apresenta a modernização tecnológica do simulador *Virtual and Interactive Distance-learning on Anatomy* (VIDA) Enfermagem, protótipo de Realidade Virtual (RV) desenvolvido em parceria entre o Laboratório de Tecnologias Interativas da USP (INTER-LAB) e a Escola de Enfermagem de Ribeirão Preto da Universidade de São Paulo (EERP-USP) para o treinamento do procedimento de PVP. A atualização envolveu a migração completa do projeto para a *Unreal Engine 5.4.4* e a substituição do *hardware* obsoleto (*Oculus Rift* e *Leap Motion*) pelo *Meta Quest 3* com rastreamento nativo de mãos via *Meta XR Interaction SDK*. Adicionalmente, implementou-se um sistema de evolução da simulação baseado em Máquina de Estados Hierárquica (MEH), estruturado em três níveis (*Storyteller*, *Stories* e *Steps*), que permite validação pedagógica das ações do usuário com *feedback* visual em tempo real através de marcadores interativos. Como resultado, obteve-se um protótipo funcional com arquitetura modular e documentação técnica completa, estabelecendo uma base sólida para desenvolvimentos futuros do simulador.

Palavras-chave: Realidade Virtual. Simulação Médica. Educação em Enfermagem.

ABSTRACT

Peripheral Venous Puncture (PVP) is a widely performed procedure in nursing practice, but students often face difficulties in its execution due to limited opportunities for supervised practice. This work presents the technological modernization of the *Virtual and Interactive Distance-learning on Anatomy* (VIDA) Nursing simulator, a *Virtual Reality* (VR) prototype developed in partnership between the *Interactive Technologies Laboratory of the University* (INTER-LAB) and *Ribeirão Preto School of Nursing* (EERP-USP) at the University of São Paulo for training in the PVP procedure. The update involved the complete migration of the project to *Unreal Engine 5.4.4* and the replacement of obsolete *hardware* (*Oculus Rift* and *Leap Motion*) with the *Meta Quest 3* featuring native hand tracking via *Meta XR Interaction SDK*. Additionally, a simulation progression system based on *Hierarchical State Machine* (HSM) was implemented, structured in three hierarchical levels (*Storyteller*, *Stories*, and *Steps*), which enables pedagogical validation of user actions with real-time visual *feedback* through interactive markers. As a result, a functional prototype was obtained with modular architecture and complete technical documentation, establishing a solid foundation for future developments of the simulator.

Keywords: Virtual Reality. Medical Simulation. Nursing Education.

LISTA DE FIGURAS

Figura 1 – Contínuo de Milgram	26
Figura 2 – Grafo Acíclico Dirigido (GAD) de uma MEH.....	30
Figura 3 – Painel <i>Outliner</i> do <i>Unreal Engine</i> mostrando a hierarquia de <i>Actors</i> no nível "Procedimento" do projeto VIDA Enfermagem.	31
Figura 4 – Editor de <i>Blueprint</i> do <i>BP_Patient</i> mostrando a estrutura de componentes e o painel de propriedades do <i>Skeletal Mesh Component</i> "remy".	32
Figura 5 – Exemplo de programação em <i>Blueprint</i>	40
Figura 6 – Unidade de exibição do Meta Quest 3.....	41
Figura 7 – Arquitetura física do sistema.	43
Figura 8 – Arquitetura funcional do sistema	45
Figura 9 – Dispositivos <i>LeapMotion</i> e <i>Oculus Rift</i> , utilizados na primeira versão do projeto VIDA Enfermagem.....	47
Figura 10 – Mensagem de erro referente à compilação de <i>plugins</i> em versão incompatível.	49
Figura 11 – Sistemas de partículas Niagara desenvolvidos para água e sabão. .	52
Figura 12 – Possíveis formas de interação disponíveis no SDK da Meta: <i>PokeInteractable</i> (toque), <i>GrabInteractable</i> (pegar) e <i>RayInteractable</i> (apontar). O sistema implementado utiliza as interações de toque e pegar.	53
Figura 13 – Fluxograma do algoritmo de progressão do sistema de evolução. ...	55
Figura 14 – Máquina de estados do <i>Storyteller</i>	56
Figura 15 – Máquina de estados de uma <i>Story</i>	56
Figura 16 – Máquina de estados de um <i>Step</i>	57
Figura 17 – Instância de esfera azul na cena do procedimento	58
Figura 18 – Capturas de tela da interface desenvolvida para o gerenciamento de âncoras e de evolução do procedimento respectivamente.	59
Figura 19 – Exemplo de <i>widget</i> de diálogo apresentado ao usuário durante etapa de identificação do paciente.	64
Figura 20 – Capturas dos feedbacks visuais de escolhas de opção de diálogo...	65
Figura 21 – Modelo do <i>widget</i> de erro desenvolvido no <i>Unreal Engine</i>	65
Figura 22 – Animações de <i>Sitting</i> e <i>Sitting Idle</i> utilizadas do Mixamo no <i>Unreal Engine Animation Editor</i>	66
Figura 23 – Máquina de Estados implementada para o movimento do paciente na simulação, com foco para a permanência no estado de <i>Sitted</i>	66
Figura 24 – Personagem <i>Remy</i> reutilizado para o personagem paciente.	67

Figura 25 –Personagem apresentado na cena do procedimento.....	68
Figura 26 –Desempenho de display da simulação com paciente	69
Figura 27 –Desempenho de GPU e CPU da simulação com paciente	70
Figura 28 –Desempenho da simulação sem o paciente	71

LISTA DE TABELAS

Tabela 4.1	–	Especificação do Requisito AMG	34
Tabela 4.2	–	Especificação do Requisito UNI	35
Tabela 4.3	–	Especificação do Requisito MQ3	35
Tabela 4.4	–	Especificação do Requisito CMP	36
Tabela 4.5	–	Especificação do Requisito AST	36
Tabela 4.6	–	Especificação do Requisito GES	37
Tabela 4.7	–	Especificação do Requisito DOC	37
Tabela 4.8	–	Especificação do Requisito EVS	38
Tabela 4.9	–	Especificação do Requisito ANC	38
Tabela 4.10	–	Especificação do Requisito FBV	38
Tabela 4.11	–	Especificação do Requisito DLG	39
Tabela 4.12	–	Especificação do Requisito VPD	39
Tabela 5.1	–	Estrutura de validação pedagógica para identificação do paciente.	61
Tabela 6.2	–	Estrutura de validação pedagógica para investigação de alergias.	87
Tabela 6.3	–	Estrutura de validação pedagógica para explicação do procedimento.	90
Tabela 6.4	–	Estrutura de validação pedagógica para obtenção do consentimento.	92
Tabela 6.5	–	Estrutura de validação pedagógica para verificação de condições pré-analíticas.	94
Tabela 6.6	–	Correspondência entre etapas do roteiro e implementação no simulador.	99

LISTA DE ABREVIATURAS E SIGLAS

EERP-USP	Escola de Enfermagem de Ribeirão Preto da Universidade de São Paulo.
GAD	Grafo Acíclico Dirigido.
HMD	<i>Head-Mounted Display</i> (<i>Display</i> montado na cabeça).
INTERLAB	Laboratório de Tecnologias Interativas da USP.
MEF	Máquina de Estados Finitos.
MEH	Máquina de Estados Hierárquica.
PMBOK	<i>Project Management Body of Knowledge</i> .
PVP	<i>Punção Venosa Periférica</i> .
RV	Realidade Virtual.
V & V	Verificação e Validação.
VIDA	<i>Virtual and Interactive Distance-learning on Anatomy</i> .

SUMÁRIO

1	INTRODUÇÃO	21
1.1	Motivação	21
1.2	Objetivos	23
1.3	Justificativa	24
1.4	Organização do Trabalho	25
2	ASPECTOS CONCEITUAIS	26
2.1	RV - Realidade Virtual	26
2.2	Punção Venosa Periférica	27
2.3	VIDA Enfermagem	27
2.4	Máquina de Estados Finitos	28
2.5	Máquina de Estados Hierárquicas	29
2.6	Conceitos Básicos da Estrutura de Jogo no <i>Unreal Engine</i>	30
3	METODOLOGIA DE TRABALHO	33
4	ESPECIFICAÇÃO DE REQUISITOS	34
5	DESENVOLVIMENTO DO PROJETO	40
5.1	Tecnologias Utilizadas	40
5.1.1	Unreal Engine	40
5.1.2	Meta Quest 3	41
5.2	Projeto e Implementação	42
5.2.1	Arquitetura do Sistema	42
5.2.1.1	Arquitetura Física	42
5.2.1.2	Arquitetura Funcional	43
5.2.2	Contexto e Impacto da Atualização Tecnológica	44
5.2.3	Recuperação e Diagnóstico do Projeto Original	49
5.2.4	Migração para Versão Atualizada da Engine	50
5.2.5	Sistema de Partículas	51
5.2.6	Interação com gestos	52
5.2.7	Sistema de Evolução da Simulação	53
5.2.8	Sistema de Âncoras	59
5.2.9	Sistema de Diálogo	61
5.2.10	Recuperação do <i>Asset</i> do Paciente na Cena do Procedimento	64
5.2.11	Testes e avaliação	67
5.2.11.1	Testes Internos	67
5.2.11.1.1	Avaliação de desempenho no <i>headset</i>	68
5.2.11.1.2	Testes Funcionais	72
6	CONSIDERAÇÕES FINAIS	76
6.1	Conclusões do Projeto de Formatura	76

6.2	Contribuições	77
6.3	Perspectivas de Continuidade	77
	REFERÊNCIAS	80
	APÊNDICES.....	84
	Apêndice A - Desafios Técnicos e Lições Aprendidas	84
	Apêndice B - Resultados dos Testes	86
	Apêndice C - Estrutura de Validação Pedagógica das Etapas de Interação	87
	Apêndice D - Descrição Completa do Procedimento de Coleta de Sangue Venoso a Vácuo	96

1 INTRODUÇÃO

1.1 Motivação

A punção venosa periférica é um procedimento amplamente realizado pelos profissionais de Enfermagem em seu cotidiano, constituindo, portanto, parte regular e comum da rotina assistencial. Ela representa cerca de 85% de todas as ações desenvolvidas pela equipe de Enfermagem (TORRES; ANDRADE; SANTOS, 2005), de forma que se estima que aproximadamente 81% desses profissionais dediquem mais de 75% de seu tempo de trabalho à condução da prática dessa intervenção (TORRES, 2003).

Segundo Nedachi (2025), as adversidades decorrentes de um procedimento de acesso periférico falho podem incluir hematomas no local da inserção, trombose do vaso sanguíneo, infecção da rede venosa, lesões nervosas e embolia gasosa, além de causarem dor e estresse ao paciente. Ela destaca, nesse contexto, que a in experiência do profissional pode contribuir também para as complicações do processo. Tais complicações tornam-se ainda mais preocupantes quando são consideradas as deficiências e lacunas na formação prática dos profissionais, conforme evidenciado por Almeida (2012).

Em estudo conduzido entre alunos do curso de Enfermagem da Fundação Presidente Antônio Carlos de Ubá, em Minas Gerais, Almeida (2012) busca avaliar o desempenho dos estudantes no que se refere ao exercício da punção venosa periférica. Ele aponta o fato de que, embora os alunos do oitavo período tenham apresentado mais confiança em realizar punções venosas periféricas do que os estudantes do sexto período, a maioria dos integrantes de ambos os grupos omitiu ou realizou incorretamente o passo da lavagem das mãos pré-punção. A etapa de uso das luvas, por exemplo, foi conduzida de forma satisfatória pelo sexto período, mas não pelo oitavo. No que se refere aos passos de comunicação com o paciente, por exemplo, como explicar o procedimento ao paciente, observar as queixas e as reações e orientar o paciente, dentre alunos do oitavo período, houve desempenho inadequado, atingindo percentuais próximos ou superiores a 50% de não realização da etapa. O estudo conclui que ambos os grupos apresentaram erros semelhantes no que diz respeito ao desempenho técnico, principalmente na comunicação com o paciente e na negligência de etapas que visam diminuir o risco de infecção. Aponta, finalmente, para a importância de aprimorar constantemente no curso de graduação o ensino dessa técnica, devido ao seu caráter rotineiro na vida do profissional da área.

A presença de erros significativos no desempenho de etapas da punção venosa

periférica não é exclusivo dos estudantes. O estudo de Torres, Andrade e Santos (2005), conduzido entre profissionais de Enfermagem na Irmandade Santa Casa de Misericórdia de Limeira, no interior paulista, apresenta conclusões similares. Nele, aponta-se o desempenho insatisfatório nas atividades de uso das luvas durante a execução do procedimento e de uso e desinfecção do torniquete. A autora sugere, portanto, que, às vezes, o nível de formação profissional pode não ter relação direta com o grau de assertividade da etapa realizada da punção.

Torres, Andrade e Santos (2005) observa que é possível que haja uma discrepância entre o conhecimento adquirido e a prática do procedimento causada por deficiências na formação profissional, principalmente no que se refere à forma de aquisição do saber e das práticas. A autora alude à construção de um sistema de reconhecimento e de retorno aos profissionais da área que valorize e reforce as condutas adequadas e esperadas durante a execução da punção.

Nesse sentido, o uso de tecnologias imersivas, como as de RV (Realidade Virtual), surge como uma alternativa complementar ao ensino teórico e prático tradicional de procedimentos de saúde. No caso da Enfermagem, a abordagem tradicional de ensino consiste na simulação do procedimento em modelos anatômicos sucedida pela prática em estudantes ou em pacientes (SOUSA et al., 2021).

Nesse contexto, as soluções de RV aprimoram o conhecimento dos estudantes e, em conjunto com outras estratégias de aprendizagem, podem ser um complemento para melhorar a qualidade e a segurança da prática clínica (CHEN et al., 2020). Sousa et al. (2021), por exemplo, avalia 6 estudos acerca de aplicações de RV na esfera da educação em saúde com destaque para o treinamento de habilidades em PVP de estudantes e profissionais da área. O autor revela que há uma melhora no conhecimento das habilidades e da prática do estudante quando são utilizados simuladores RV para a prática de PVP ao invés da metodologia tradicional de ensino, como pode ser atestado pela redução do número de reinserções no paciente e do tempo de execução do procedimento, por exemplo.

Sendo assim, os avanços em desenvolvimento e aplicações de tecnologias de RV têm acarretado em novas oportunidades para o treinamento de estudantes da área da saúde. O uso de simuladores de RV no ensino de procedimentos médicos tem se mostrado um método promissor para o desenvolvimento de habilidades práticas em um ambiente controlado e seguro antes da interação com pacientes reais, pois oferece um espaço seguro sem risco de vida em função do uso dos equipamentos e passível a erros e repetições, o que deixa os estudantes mais confiantes durante a prática da PVP (SOUSA et al., 2021).

O projeto VIDA Enfermagem é um protótipo de simulador de RV de PVP de-

envolvido pelo INTERLAB em parceria com a EERP-USP com o intuito de melhorar o aprendizado dos estudantes dessa escola em relação ao procedimento de coleta de sangue. Este projeto de formatura é uma evolução do trabalho de pesquisa de Souza-Júnior (2018), que resultou no desenvolvimento do protótipo inicial do simulador, com foco nas habilidades práticas dos estudantes de Enfermagem. O simulador representa um passo importante não somente no avanço das pesquisas sobre RV aplicada à PVP, que ainda são escassas a despeito do crescente interesse a respeito do uso de tecnologias imersivas na área da saúde, mas também na evolução da participação brasileira no rol desses estudos, que é liderado pelos Estados Unidos (SOUSA et al., 2021).

Embora existam demandas de melhorias e adições ao sistema VIDA Enfermagem, seu desenvolvimento técnico não foi atualizado nos últimos anos. Como resultado, existem atualmente dificuldades de atualização e mesmo de implantação do sistema, decorrentes de incompatibilidades tecnológicas entre as versões do protótipo e os ambientes computacionais atuais.

1.2 Objetivos

À luz do exposto, o objetivo geral deste trabalho é atualizar tecnologicamente o simulador VIDA Enfermagem, desenvolvendo um novo projeto de RV que garanta sua execução correta em diferentes ambientes e estabeleça uma base sólida para desenvolvimentos futuros.

Os objetivos específicos são:

- Realizar a migração do protótipo para o *Head-Mounted Display* (*Display* montado na cabeça) (HMD) Meta Quest 3 e para a Unreal Engine 5.4.4, aproveitando os avanços tecnológicos recentes em RV;
- Implementar um subconjunto controlado de etapas básicas da simulação de PVP, estabelecendo uma base funcional para expansões futuras;
- Unificar as versões do protótipo existentes nos laboratórios parceiros, garantindo portabilidade entre diferentes sistemas operacionais e dispositivos;
- Desenvolver documentação técnica completa do sistema, facilitando sua manutenção e evolução por futuros desenvolvedores;
- Validar o novo protótipo junto à EERP-USP, assegurando que as atualizações mantenham a fidelidade ao procedimento real e às necessidades educacionais.

Tais avanços contribuirão para a complementação da formação de enfermeiros

mais preparados e capacitados, com um entendimento robusto dos procedimentos de PVP, antes de realizá-los em pacientes reais. Ao promover uma nova etapa de evolução e consolidar a vitalidade do protótipo VIDA, este trabalho pretende favorecer a integração entre a metodologia tradicional de ensino e as simulações de RV.

Cabe ressaltar que este trabalho concentra-se na modernização tecnológica e na consolidação arquitetônica do simulador existente, estabelecendo uma base robusta e bem documentada para desenvolvimentos futuros. A implementação de todas as etapas detalhadas do procedimento completo de PVP, bem como testes de usabilidade em larga escala com múltiplas turmas, são considerados evoluções naturais que se beneficiarão da base tecnológica moderna estabelecida por este trabalho.

1.3 Justificativa

O protótipo VIDA Enfermagem passou por processo de validação de aparência e de conteúdo testada por dois grupos formados por 15 integrantes cada (SOUZA-JÚNIOR et al., 2020). O primeiro deles foi composto por profissionais da área da saúde com *expertise* no procedimento de PVP e o segundo formado por graduandos do curso de Enfermagem. O resultado da avaliação desses participantes aponta que o protótipo VIDA Enfermagem é uma ferramenta promissora e válida como estratégia complementar de ensino do processo de PVP a vácuo em pacientes adultos.

Apesar dos resultados promissores, o protótipo atual enfrenta desafios significativos que limitam sua aplicação prática e escalabilidade. O projeto encontra-se atualmente em distintas versões não unificadas, utilizadas pelos laboratórios parceiros e sujeitas a problemas de execução em diferentes sistemas operacionais e dispositivos. Além disso, há lacunas importantes na documentação técnica do sistema, o que dificulta sua manutenção e evolução por novos desenvolvedores. A tecnologia de *hardware* e *software* utilizada no protótipo original também se encontra defasada em relação aos avanços recentes em RV, como dispositivos de maior resolução e *engines* gráficas mais robustas.

Nesse contexto, este trabalho se justifica pela necessidade de atualizar tecnologicamente o simulador, garantir sua portabilidade e uniformização entre diferentes ambientes de execução, e estabelecer uma base sólida e bem documentada para desenvolvimentos futuros. A atualização para o HMD Meta Quest 3 e para a Unreal Engine 5.4.4 não apenas incorpora melhorias significativas em qualidade gráfica e desempenho, mas também garante maior longevidade ao projeto ao utilizar tecnologias de ponta amplamente suportadas pela comunidade de desenvolvimento.

Além disso, o trabalho proposto atende uma deficiência existente na área de

treinamentos imersivos, haja vista a escassez de publicações sobre o tema (SOUSA et al., 2021) e a importância do uso de ferramentas de simulações em RV como complemento às metodologias tradicionais de ensino. Ao consolidar e modernizar o protótipo existente, este projeto contribui tanto para o avanço das pesquisas nacionais em RV aplicada à saúde quanto para a formação de enfermeiros mais preparados e capacitados, com um entendimento robusto dos procedimentos de PVP antes de realizá-los em pacientes reais.

O desenvolvimento deste trabalho conta com a colaboração da professora Simone de Godoy Costa, da EERP-USP, tanto no levantamento de requisitos quanto na validação do sistema, garantindo que as atualizações implementadas mantenham a fidelidade ao procedimento real e às necessidades educacionais dos estudantes de Enfermagem.

1.4 Organização do Trabalho

Este relatório está organizado em 6 capítulos. O primeiro capítulo expõe a motivação, os objetivos e a justificativa de relevância deste trabalho de conclusão de curso para a área de simulação de treinamento para o ensino da saúde.

O segundo capítulo aborda a base teórica de suporte ao projeto e conceitos essenciais para a compreensão das atividades do projeto. No terceiro capítulo, apresentam-se as ferramentas e os recursos utilizados para o desenvolvimento do sistema.

O quarto capítulo apresenta um detalhamento sobre as atividades do fluxo de desenvolvimento do projeto, enquanto o quinto capítulo especifica os requisitos a serem implementados.

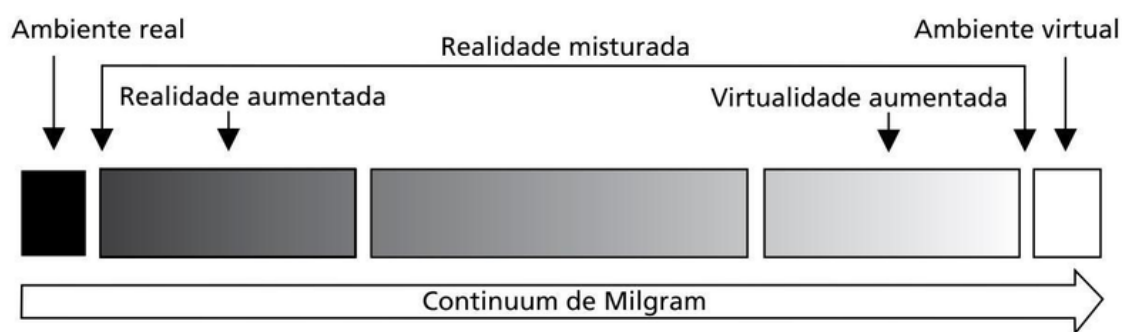
Por fim, o último capítulo apresenta a distribuição dos requisitos e demais atividades de projeto ao longo do período de referência, a partir do mapeamento das prioridades dos requisitos.

2 ASPECTOS CONCEITUAIS

2.1 RV - Realidade Virtual

O conceito de RV pode ser compreendido dentro do Contínuo de Milgram – ou Contínuo Real-Virtual, proposto por Milgram et al. (1995). Este modelo considera ambientes reais e virtuais não como conceitos antitéticos, mas sim extremos opostos de um contínuo que podem eventualmente se misturar. Sendo assim, ele introduz uma escala contínua para representar a passagem do completamente real para o completamente virtual e vice-versa (Fig. 1).

Figura 1: Contínuo de Milgram



Fonte: Adaptado de Milgram et al. (1995) por Tori e Hounsell (2018).

No extremo direito, situa-se a RV, ou, a virtualidade, o completamente virtual, enfim, o ambiente virtual, ao passo que a realidade, ou o completamente real, o ambiente real, posiciona-se no extremo esquerdo deste espectro. A interface entre ambos esses espaços é o marco mais importante deste modelo, no que se refere a RV, e é nomeada de Realidade Misturada, sendo composta tanto pela Realidade Aumentada como pela Virtualidade Aumentada.

No que se refere a isso, a RV relaciona-se com dois conceitos importantes: imersão e presença. O primeiro pode ser compreendido como o nível de precisão da ilusão de uma realidade distinta fornecida por um sistema computacional a seu usuário e pode ser caracterizada por variáveis como a qualidade da imagem, campo de visão, interatividade, abrangência e enredo. O segundo, por sua vez, pode ser entendido como a percepção subjetiva do usuário de que o ambiente virtual é real e relaciona-se às ilusões de presença espacial, corporal, física e social (TORI; HOUNSELL, 2018).

Diante disso, em consonância com este trabalho, é conveniente definir RV como um uma experiência completamente imersiva executada no computador na qual o

usuário se abstrai do mundo tangível para interagir em tempo real com elementos de uma simulação tridimensional de um ambiente ou de uma situação real.

2.2 Punção Venosa Periférica

O processo de punção venosa periférica consiste na “inserção de dispositivos por veia periférica para acesso à corrente sanguínea” (SOUZA-JÚNIOR et al., 2020) após assepsia do local escolhido. Fatores como o estado do paciente, visibilidade, acessibilidade e palpabilidade podem influenciar a decisão de qual região do braço ou da mão será escolhida para o procedimento.

Essa técnica é utilizada com bastante frequência nos ambientes hospitalares e laboratoriais com fins de administração de medicamentos ou de líquidos intravenosos, como soro, e até mesmo para realizar coleta de sangue. Sendo assim, não é surpreendente que seja um dos procedimentos mais comuns realizados pelos estudantes de Enfermagem como introdução à prática clínica.

Soma-se a isso a possibilidade de implicações negativas no paciente diante da execução inadequada da punção. Dentre elas, vale destacar os hematomas, a flebite, as infecções e, até mesmo, os resultados incorretos de exames, se for o caso. Dessa forma, torna-se ainda mais importante o aprendizado da condução correta deste procedimento médico e a perícia por parte do profissional que o realiza.

A descrição detalhada de todas as etapas que compõem o procedimento pode ser consultada no Apêndice 6.3.

2.3 VIDA Enfermagem

O sistema VIDA Enfermagem é uma parceria realizada entre o INTERLAB e a EERP-USP que consiste no desenvolvimento de um simulador de RV imersiva do procedimento de punção venosa periférica em pacientes adultos com o objetivo de complementar o treinamento de estudantes de Enfermagem no que se refere ao exercício da coleta de sangue à vácuo.

O projeto VIDA, por sua vez, também foi desenvolvido pelo INTERLAB, com o intuito de construir um atlas virtual de aprendizagem para treinamento de estudantes de cursos de áreas da saúde. Nessa ferramenta, o usuário interage com modelos tridimensionais de estruturas anatômicas, acessados pelo seu próprio computador com o auxílio apenas de um óculos para visualização estéreo e de sua câmera web (Tori et al., 2009).

Sendo assim, VIDA Enfermagem se une a outros esforços como VIDA Odonto, um simulador de RV destinado ao treinamento de procedimentos odontológicos (Tori et al., 2016), e VIDA XR, uma aplicação de realidade estendida para manipulação de modelos tridimensionais de dentes humanos (Arcanjo et al., 2024), como evoluções e braços do projeto VIDA.

A primeira versão do simulador VIDA Enfermagem, nomeada VIDA-Enfermagem 1.0, foi desenvolvida e validada por Souza-Júnior et al. (2020). Ela foi projetada considerando 16 etapas do procedimento, com foco na interatividade e precisão dos movimentos, e contou com a participação de uma equipe multidisciplinar formada por engenheiros mecânicos e elétricos, enfermeiros e graduandos tanto de engenharia como de enfermagem nas etapas de modelagem 3D, interação e programação da ferramenta. Diante disso, as atividades deste projeto passam a se concentrar em um recorte selecionado dessas etapas, implementando uma versão básica dos passos escolhidos, centrada no núcleo do procedimento, sem abranger a totalidade do procedimento original.

2.4 Máquina de Estados Finitos

As Máquina de Estados Finitos (MEF) são uma ferramenta matemática amplamente utilizada na área da Computação para a modelagem de sistemas dinâmicos cujo comportamento pode ser descrito por um conjunto finito de configurações. Formalmente, uma MEF pode ser definida como uma sêxtupla $M = (S, I, O, f, g, s_0)$, onde S representa o conjunto finito de estados, I é o alfabeto finito de entrada, O é o alfabeto finito de saída, $f : S \times I \rightarrow S$ é a função de transição que determina o próximo estado, $g : S \times I \rightarrow O$ é a função de saída, e $s_0 \in S$ é o estado inicial (ROSEN, 2019).

De forma tradicional, uma MEF é definida por meio de uma função de transição de estado, responsável por determinar o próximo estado a partir do par composto pelo estado atual e pela entrada disponível em um determinado instante. Além disso, ela produz saídas associadas ao seu funcionamento interno, que podem representar ações executadas pelo sistema, mensagens exibidas ao usuário ou mudanças no estado visual do ambiente, quando aplicadas no contexto de RV.

A representação gráfica mais comum para uma MEF são os diagramas de estados. Nesse tipo de diagrama, cada estado possível do sistema é indicado por círculos ou caixas, ao passo que as transições são setas dirigidas. As convenções para o formato de anotações variam na literatura, mas é comum o uso de E/S acima das setas, representando a entrada necessária para disparar a transição e a saída produzida após sua execução, respectivamente. Estados iniciais são tipicamente indicados por

uma seta sem origem.

No contexto de aplicações interativas e sistemas de RV, as MEF desempenham papel fundamental na modelagem de diferentes aspectos do sistema. Elas são comumente utilizadas para controlar o comportamento de personagens virtuais, definindo suas reações a estímulos do usuário ou do ambiente. Além disso, MEF são empregadas na modelagem de interfaces de usuário, gerenciando transições entre diferentes modos de interação, menus e telas. Em simulações educacionais, como no caso do treinamento de procedimentos clínicos, MEF permitem a validação sequencial de etapas, garantindo que o usuário execute ações na ordem correta antes de prosseguir para fases subsequentes do procedimento.

Apesar de serem simples, as MEF tradicionais apresentam algumas limitações ao modelar sistemas complexos. O número de estados, por exemplo, pode crescer significativamente com a complexidade do sistema, o que resulta em diagramas mais difíceis de compreender. Além disso, a falta de uma estrutura hierárquica dificulta a reutilização de componentes e a modularização do sistema (YANNAKAKIS, 2000).

2.5 Máquina de Estados Hierárquicas

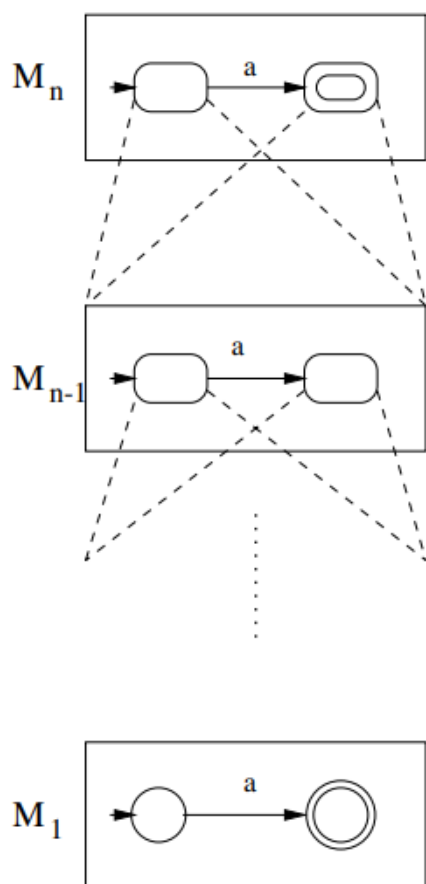
As Máquinas de Estado Hierárquicas (MEH) são uma extensão das MEF em que cada estado pode ser tanto um estado atômico como uma máquina de estados completa. Essa característica permite a criação de estruturas aninhadas com múltiplos níveis de abstração, possibilitando a decomposição recursiva de comportamentos complexos em componentes menores e mais gerenciáveis (YANNAKAKIS, 2000).

Formalmente, uma MEH pode ser definida de forma indutiva. No caso base, toda MEF básica é também uma MEH. Recursivamente, uma nova MEH pode ser formada a partir do mapeamento dos estados de uma MEF para outras máquinas previamente definidas (YANNAKAKIS, 2000).

Uma representação gráfica para MEH utiliza Grafos Acíclicos Direcionados (GAD), nos quais cada folha corresponde a uma MEF. Dentro de cada nó u dessas máquinas, pode haver um mapeamento μ_u que conecta u a uma submáquina M_u do grafo, criando assim a estrutura hierárquica, como ilustra a Figura 2

Essa estrutura hierárquica oferece vantagens significativas sobre MEF tradicionais na modelagem de sistemas complexos. Primeiramente, a modularidade permite a reutilização de submáquinas em diferentes contextos sem necessidade de replicação de estados, o que facilita a manutenção e a evolução do sistema. Além disso, a hierarquia reduz consideravelmente a explosão de estados característica de MEF tra-

Figura 2: GAD de uma MEH.



Fonte: Yannakakis (2000).

dicionais em sistemas complexos, uma vez que comportamentos relacionados podem ser encapsulados em submáquinas diferentes e independentes. A estrutura em camadas, por fim, torna o sistema mais compreensível, permitindo que diferentes níveis de abstração sejam analisados separadamente conforme a necessidade.

Em conjunto, essas características tornam as MEH particularmente mais adequadas para modelagem de sistemas interativos que possuem lógicas que exigem decomposição de procedimentos complexos em etapas sequenciais validáveis, como é o caso de simulações educacionais em RV.

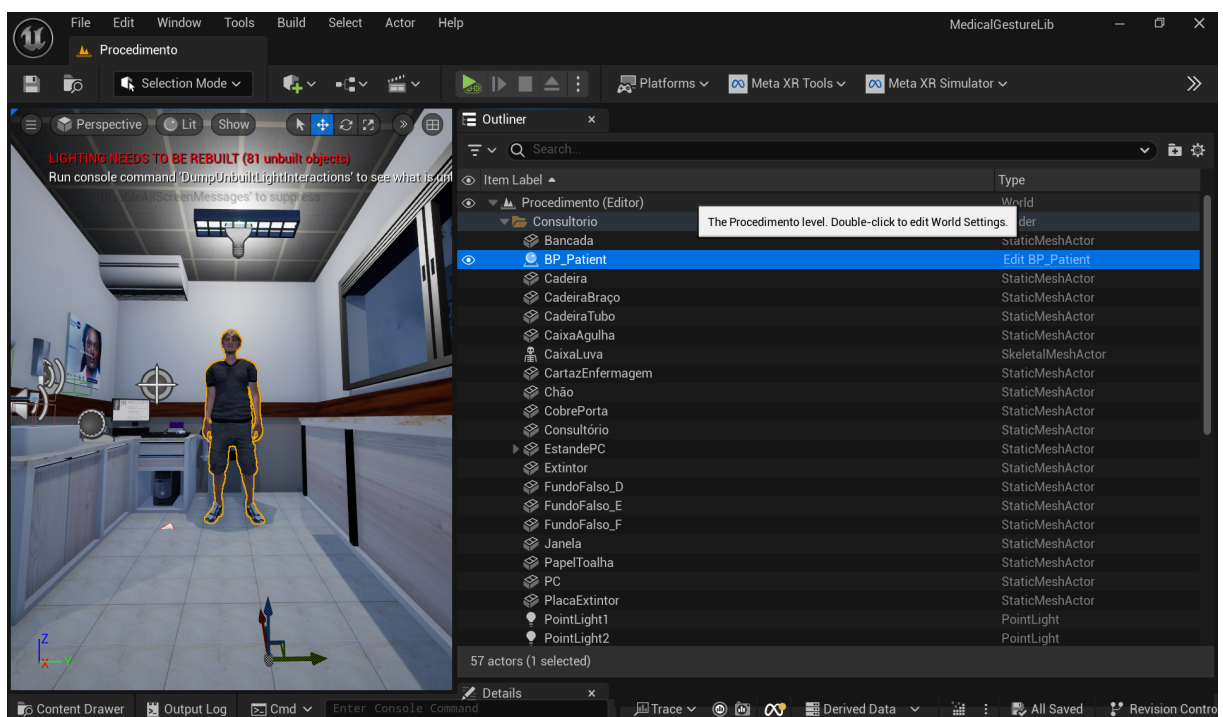
2.6 Conceitos Básicos da Estrutura de Jogo no *Unreal Engine*

No *Unreal Engine*, um nível ou cena corresponde aos ambientes da simulação. Um *Actor* é qualquer objeto posicionado na cena, como personagens, equipamentos ou câmeras. Cada *Actor* possui *Components* que definem suas funcionalidades. Um componente de malha esquelética (*Skeletal Mesh Component*), por exemplo, define a

representação visual e estrutura para animações de personagens.

A Figura 3 ilustra a hierarquia de objetos no editor de nível do *Unreal Engine*, mostrando como *Actors* são organizados dentro do nível "Procedimento". No painel *Outliner*, é possível observar diversos *Actors* que compõem a cena da simulação, incluindo o *BP_Patient* (personagem paciente), objetos estáticos como cadeira e bancada, e elementos de iluminação. Cada *Actor* listado possui um tipo específico (*StaticMeshActor*, *SkeletalMeshActor*, *PointLight*), que define suas características fundamentais.

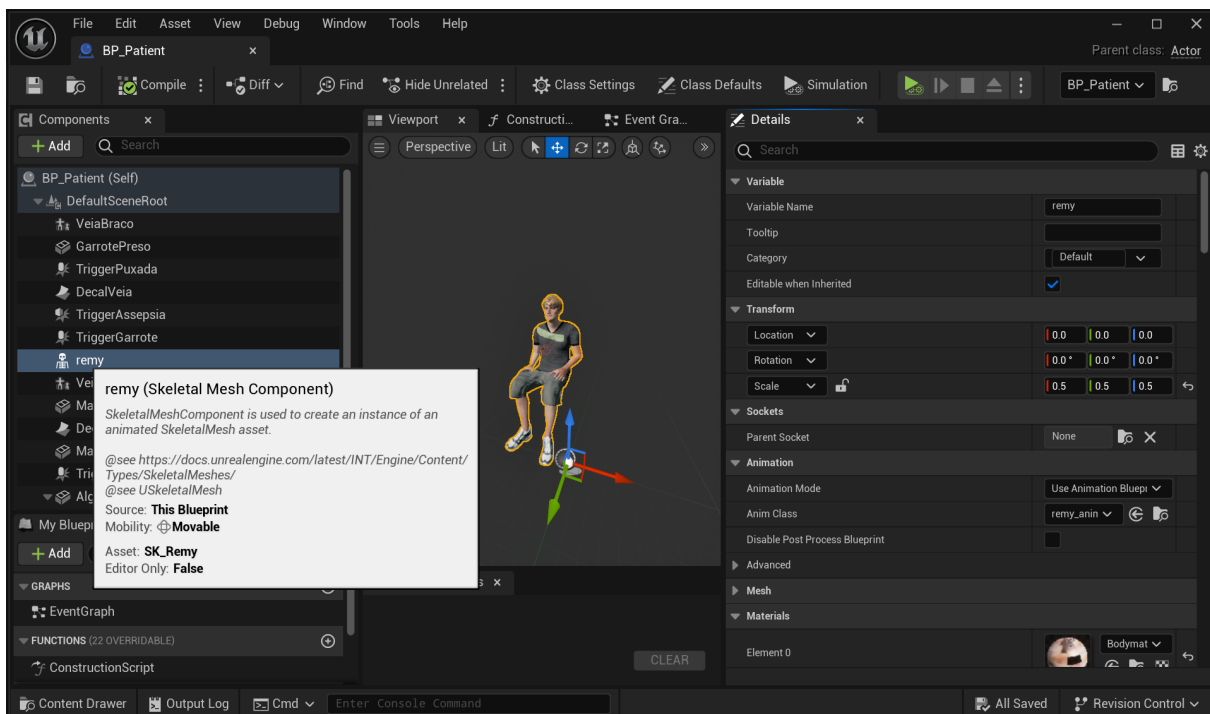
Figura 3: Painel *Outliner* do *Unreal Engine* mostrando a hierarquia de *Actors* no nível "Procedimento" do projeto VIDA Enfermagem.



Fonte: Capturado pelos autores.

A Figura 4 apresenta a estrutura interna do *Actor BP_Patient* no editor de *Blueprint*. No painel *Components* à esquerda, observa-se a hierarquia de componentes que compõem o personagem, incluindo *DefaultSceneRoot* (raiz da hierarquia), *remy* (*Skeletal Mesh Component* responsável pela representação visual), e outros componentes funcionais como *VeiaBraco*, *GarrotePreso* e diversos *triggers* de interação. O painel *Details* à direita exibe as propriedades configuráveis do componente *remy*, como malha esquelética associada (*SK_Remy*), classe de animação (*remy_anim*), e configurações de transformação espacial. Essa estrutura modular permite que funcionalidades distintas sejam encapsuladas em componentes específicos, facilitando manutenção e reutilização.

Figura 4: Editor de *Blueprint* do *BP_Patient* mostrando a estrutura de componentes e o painel de propriedades do *Skeletal Mesh Component* "remy".



Fonte: Capturado do *Unreal Engine 5.4.4* pelos autores.

3 METODOLOGIA DE TRABALHO

A organização das atividades do projeto baseia-se em quatro etapas de desenvolvimento: levantamento de requisitos, seleção de escopo, implementação e Verificação e Validação (V & V). Durante a primeira etapa, a realização de discussões para compreensão da simulação com os colaboradores da EERP-USP deve guiar o processo de elaboração das funcionalidades e das restrições desejadas para o simulador, o que contribui para a delimitação de uma solução adequada.

Por conseguinte, deve-se conduzir um estudo detalhado dos resultados obtidos com trabalhos anteriores envolvendo o projeto VIDA Enfermagem (SOUZA-JÚNIOR, 2018), a partir da análise da documentação existente, tanto na forma de repositórios de código como na forma de artigos científicos. À luz das informações obtidas, então, prossegue-se com a enumeração dos requisitos necessários, associados às suas respectivas prioridades. Por fim, espera-se obter uma lista final com as especificações dos requisitos a serem implementados neste trabalho, de acordo com os recursos de tempo disponíveis.

Diante disso, é iniciado o ciclo de iterações de implementação do protótipo. Ele é composto pela repetição das três etapas seguintes:

1. Um recorte dos requisitos é escolhido, desenvolvido e direcionado aos colaboradores da EERP;
2. Colaboradores da EERP submetem a versão do projeto com o requisito implementado para V & V realizado pelos seus alunos, enquanto o ciclo seguinte é iniciado;
3. Com as avaliações obtidas, correções e melhorias são introduzidas continuamente ao projeto, até sua conclusão.

4 ESPECIFICAÇÃO DE REQUISITOS

Após a deliberação com as partes interessadas da EERP e o estudo do material previamente implementado, foi conduzida a seleção e a caracterização de requisitos, conforme proposto por Oberg, Probasco e Ericsson (2001). As informações elencadas compreendem:

- Código de Identificação;
- Nome;
- Descrição;
- Prioridade;
- Estabilidade;
- *Rationale* (informações adicionais);
- Requisitos associados.

Os onze requisitos especificados para o projeto são apresentados entre a Tabela 4.1 e a Tabela 4.12.

Tabela 4.1: Especificação do Requisito AMG

Código:	AMG	[] Funcional [x] Não Funcional
Requisito:	Atualização do motor gráfico	
Descrição:	Migrar o projeto desenvolvido em Unreal Engine 4 para Unreal Engine 5, garantindo que o protótipo possa ser aberto e compilado sem erros críticos.	
Prioridade:	[x] Alta [] Média [] Baixa	
Estabilidade:	[x] Alta [] Média [] Baixa	
<i>Rationale:</i>	Etapa de atualização do projeto	
Requisitos Associados:	MQ3, AST, CMP	

Tabela 4.2: Especificação do Requisito UNI

Código:	UNI	☐ Funcional ☒ Não Funcional
Requisito:	Uniformização do protótipo	
Descrição:	Garantir que o projeto migrado abra e funcione corretamente em diferentes computadores, sem gerar falhas ou <i>crashes</i> .	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	O protótipo presente no INTERLAB e utilizado pela EERP não somente apresentava falhas críticas ao ser aberto em máquinas diferentes, dificultando testes, desenvolvimento colaborativo e validação, mas também, com frequência, haviam múltiplas distintas versões em ambos os laboratórios disponíveis, o que dificultava a sua execução e reprodutibilidade.	
Requisitos Associados:		

Tabela 4.3: Especificação do Requisito MQ3

Código:	MQ3	☐ Funcional ☒ Não Funcional
Requisito:	Compatibilidade com Meta 3	
Descrição:	Remover suporte para a linha Oculus Rift e adicionar suporte para a linha de HMD Meta Quest 3.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Etapa de atualização do projeto.	
Requisitos Associados:	AMG, AST, CMP	

Tabela 4.4: Especificação do Requisito CMP

Código:	CMP	☐ Funcional ☒ Não Funcional
Requisito:	Compatibilidade de plugins	
Descrição:	Realizar a integração entre os Blueprints do protótipo e os plugins de rastreamento de mãos utilizados, garantindo que estes sejam compatíveis e totalmente operacionais na versão atualizada do Unreal Engine 5, substituindo o dispositivo Leap Motion descontinuado.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Etapa de atualização do projeto	
Requisitos Associados:	AMG, MQ3, AST	

Tabela 4.5: Especificação do Requisito AST

Código:	AST	☐ Funcional ☒ Não Funcional
Requisito:	Preservação de assets	
Descrição:	Assegurar que os principais <i>assets</i> (do ambiente, do procedimento e do paciente) utilizados no protótipo (modelos, texturas, animações) mantenham sua funcionalidade e integridade na versão migrada, sem haver corrupção dos <i>assets</i> .	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Etapa de atualização do projeto	
Requisitos Associados:	AMG, MQ3, CMP	

Tabela 4.6: Especificação do Requisito GES

Código:	GES	[x] Funcional [] Não Funcional
Requisito:	Rastreamento das mãos	
Descrição:	Reconhecer gestos válidos realizados pelo usuário com alta confiabilidade através do uso do kit de interações da Meta (META, 2025c). A detecção deve ocorrer de forma contínua, com baixa latência e tolerância a pequenas variações na posição ou orientação das mãos.	
Prioridade:	[x] Alta [] Média [] Baixa	
Estabilidade:	[x] Alta [] Média [] Baixa	
Rationale:	Garantir que a experiência seja imersiva e responsiva. Quando reconhecer uma pose definida previamente, o sistema deve reagir de forma quase imediata, acionando uma animação ou uma interação correspondente entre os blueprints presentes na cena. Isso é essencial para manter a naturalidade das interações	
Requisitos Associados:		

Tabela 4.7: Especificação do Requisito DOC

Código:	DOC	[] Funcional [x] Não Funcional
Requisito:	Documentação	
Descrição:	Garantir que todo o projeto, incluindo código-fonte, as-sets, configurações e etapas de desenvolvimento, esteja devidamente documentado e versionado em fer-ramentas como GitHub, README detalhado, change-logs e arquivos de instrução, permitindo continuidade e recuperação de informações importantes.	
Prioridade:	[x] Alta [] Média [] Baixa	
Estabilidade:	[x] Alta [] Média [] Baixa	
Rationale:	Informações críticas do projeto foram perdidas ou es-tão dispersas; uma documentação centralizada e ver-sionada garante que futuras atualizações, correções e testes possam ser realizados sem perda de dados ou retrabalho.	
Requisitos Associados:		

Tabela 4.8: Especificação do Requisito EVS

Código:	EVS	☒ Funcional ☐ Não Funcional
Requisito:	Sistema de progressão de etapas	
Descrição:	Implementar sistema que gerencia a sequência de execução das etapas do procedimento, validando ações do usuário e controlando transições entre estados.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Estrutura modular que permite adição de novas etapas sem reconfiguração completa do sistema, refletindo a natureza sequencial do procedimento de PVP.	
Requisitos Associados:	GES	

Tabela 4.9: Especificação do Requisito ANC

Código:	ANC	☒ Funcional ☐ Não Funcional
Requisito:	Posicionamento dinâmico de indicadores visuais	
Descrição:	Permitir que marcadores visuais sejam posicionados dinamicamente na cena através de pontos de referência identificáveis, independentes do layout físico do ambiente.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Facilita ajustes na disposição dos objetos da cena sem necessidade de reconfiguração da lógica do procedimento.	
Requisitos Associados:	EVS	

Tabela 4.10: Especificação do Requisito FBV

Código:	FBV	☒ Funcional ☐ Não Funcional
Requisito:	Feedback visual de validação	
Descrição:	Fornecer indicadores visuais diferenciados para etapas disponíveis, ações corretas e erros cometidos, acompanhados de mensagens explicativas quando apropriado.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
Rationale:	Orienta o usuário através das etapas em tempo real, permitindo aprendizado com correção imediata de erros.	
Requisitos Associados:	EVS, ANC	

Tabela 4.11: Especificação do Requisito DLG

Código:	DLG	☒ Funcional ☐ Não Funcional
Requisito:	Interação dialogada com paciente virtual	
Descrição:	Permitir interação estruturada entre estudante e paciente virtual através de perguntas e respostas com validação pedagógica e feedback diferenciado.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
<i>Rationale:</i>	Simula etapas de comunicação com paciente essenciais ao procedimento, promovendo desenvolvimento de competências de raciocínio clínico e tomada de decisão.	
Requisitos Associados:	EVS, GES	

Tabela 4.12: Especificação do Requisito VPD

Código:	VPD	☐ Funcional ☒ Não Funcional
Requisito:	Conformidade pedagógica com protocolos clínicos	
Descrição:	Garantir que conteúdo de diálogo, perguntas, respostas e justificativas sigam protocolos de enfermagem e diretrizes de segurança validados pela EERP-USP.	
Prioridade:	☒ Alta ☐ Média ☐ Baixa	
Estabilidade:	☒ Alta ☐ Média ☐ Baixa	
<i>Rationale:</i>	Assegura fidelidade ao procedimento real e alinhamento com necessidades educacionais dos estudantes de Enfermagem.	
Requisitos Associados:	DLG	

5 DESENVOLVIMENTO DO PROJETO

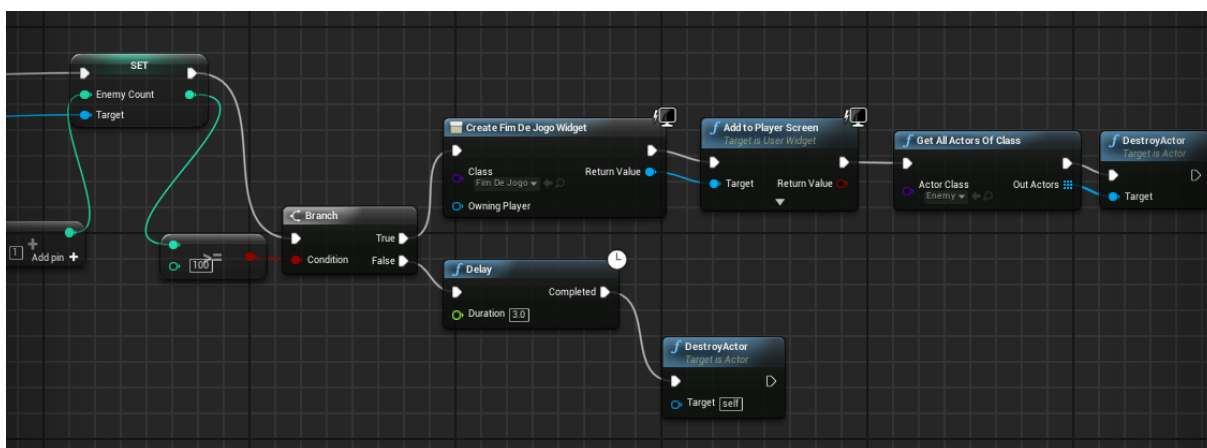
5.1 Tecnologias Utilizadas

5.1.1 Unreal Engine

O *Unreal Engine* é um motor gráfico de desenvolvimento de jogos e simulações criado pela *Epic Games* e que ganhou popularidade por ser uma plataforma que possibilita a criação de jogos, produção e animação de filmes, interfaces arquitetônicas e automobilísticas, etc., de forma bastante simplificada e com recursos avançados. É uma ferramenta que foi utilizada na construção de jogos de séries famosas como *Batman Arkham* (ENGINE, 2012), *Mortal Kombat* (GAUDIOSI, 2015), *Street Fighter* (GAUDIOSI, 2016) e até mesmo *Fortnite* (ENGINE,).

A plataforma permite utilizar tanto programação pura em C++ como programação em *Blueprint* para a criação de produtos. Nesse sentido, o *Blueprint Visual Scripting* (Fig. 5) corresponde a um sistema completo de script visual que se baseia no conceito de interface em nós para criar lógicas e elementos de gameplay dentro do *Unreal*, arrastando e conectando nós (GAMES, 2025).

Figura 5: Exemplo de programação em *Blueprint*



Fonte: Autoria própria.

Isso facilita o desenvolvimento de jogos, pois permite que equipes sem domínio técnico de linguagens de programação possam trabalhar em projetos dentro desse ecossistema, e, por conta disso, é a alternativa mais popular de trabalho com *Unreal*. O protótipo do projeto VIDA Enfermagem foi desenvolvido utilizando o sistema de programação *Blueprint* e, por motivos de continuidade, optou-se por manter esta escolha para este trabalho.

5.1.2 Meta Quest 3

O Meta Quest 3 (Fig. 6) é a terceira geração do Meta Quest, linha de headsets de RV desenvolvido pela Meta. Este modelo de HMD apresenta alguns avanços diferenciais das gerações anteriores, como o uso do processador Snapdragon XR2 Gen 2, que proporciona o dobro de poder de processamento e uma experiência de jogabilidade mais fluida e expressiva para o usuário; o armazenamento de 512GB; o alcance de áudio 40% mais alto e 33% a mais de DRAM que a geração anterior (META, 2025a).

Figura 6: Unidade de exibição do Meta Quest 3



Fonte: Commons (2023)

Ele ainda conta com uma resolução aprimorada de 2064x2208 pixels por olho, o que possibilita uma visualização mais imersiva, nítida e de alta fidelidade das imagens, além de um sistema de Realidade Misturada mais avançado composto por 2 câmeras RGB que conferem uma resolução de *passthrough* colorido cerca de 5 vezes maior que o modelo anterior e por um projetor de profundidade que proporciona maior qualidade de rastreamento de profundidade e de resolução.

O sistema de lentes *Pancake Lens* apresenta lentes melhoradas, mais finas e mais leves, que aumentam também a nitidez e reduzem o tamanho do headset. Isso proporciona uma melhor distribuição do peso do HMD e pode diminuir o cansaço com o longo tempo de seu uso. Este modelo, por fim, apresenta compatibilidade com versões anteriores, ou seja, é possível continuar jogando jogos do Meta Quest 2 no Meta Quest 3.

O HMD acompanha um adaptador universal do tipo USB-C e 2 controles *Touch Plus*, que contam com um design mais ergonômico e simplificado, sem anéis de ras-

treamento e com tecnologia háptica aprimorada que fornece feedback tátil para experiência mais realista.

O protótipo do VIDA Enfermagem foi desenvolvido voltado ao conjunto Oculus Rift e *Touch Virtual Reality System*, isto é, um *Head Mounted Display* e um Controle, e a escolha para o uso do Meta Quest 3 na versão do projeto desenvolvida neste trabalho parte da necessidade de dispensar uma tecnologia obsoleta e descontinuada. O Meta Quest 3, além de ser uma ferramenta mais recente, está disponível no INTERLAB e seu uso no projeto apresenta diversas vantagens, à luz do exposto, como qualidade de imagem superior, feedback háptico, desempenho e processamento melhores, maior mobilidade, etc.

5.2 Projeto e Implementação

Nesta seção, são apresentadas informações referentes às arquiteturas física e funcional do sistema, com uma discussão aprofundada sobre as alterações necessárias para atualização tecnológica do sistema.

5.2.1 Arquitetura do Sistema

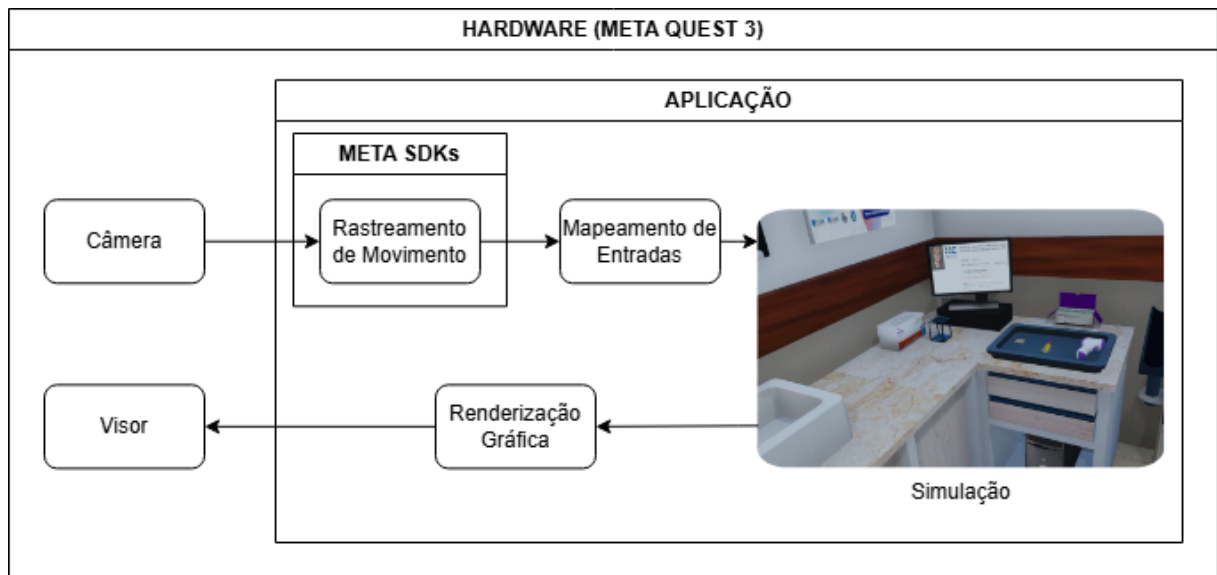
5.2.1.1 Arquitetura Física

A arquitetura do sistema foi definida considerando a integração entre os componentes físicos e lógicos da aplicação. A Figura 7 representa a arquitetura física do sistema, que utiliza o modelo *standalone*, no qual o dispositivo Meta Quest 3 funciona de forma autônoma, sem necessidade de processamento externo.

A arquitetura é composta por duas camadas principais. A camada de **Hardware** é representada pelo Meta Quest 3, que integra câmeras para rastreamento de movimento e visor para renderização gráfica. As câmeras capturam os movimentos do usuário e do ambiente, enquanto o visor exibe o conteúdo visual renderizado em três dimensões.

A camada de **Aplicação**, por sua vez, processa os dados capturados pelas câmeras através dos *Meta SDKs*, que fornecem funcionalidades de rastreamento de movimento e mapeamento de entradas gestuais. Esses dados são então utilizados pela aplicação desenvolvida em Unreal Engine 5.4.4, que implementa a simulação do procedimento de PVP. A aplicação processa as interações do usuário, gerencia a lógica do sistema de evolução de passos e prepara os elementos visuais para renderização.

Figura 7: Arquitetura física do sistema.



Fonte: Elaborado pelos autores.

O fluxo de processamento ocorre da seguinte forma: as câmeras capturam os movimentos do usuário, os *Meta SDKs* processam esses dados para identificar gestos e posições, a aplicação interpreta essas informações de acordo com a lógica da simulação e gera os elementos visuais correspondentes, que são então renderizados graficamente e exibidos no visor do HMD. Este ciclo contínuo garante uma experiência imersiva e responsiva.

Vale ressaltar que o dispositivo é também responsável pelo armazenamento local da aplicação, operando de forma completamente autônoma sem dependência de servidores externos ou computadores auxiliares. Essa arquitetura *standalone* modular e integrada permite uma experiência de RV robusta em um ambiente responsivo, com foco na imersão do usuário durante o treinamento.

5.2.1.2 Arquitetura Funcional

A arquitetura funcional do sistema, ilustrada na Figura 8, demonstra a integração entre os principais componentes desenvolvidos e o fluxo de interação do usuário durante a simulação. O sistema é estruturado em três camadas principais que trabalham de forma coordenada: entrada do usuário, gerenciamento de etapas e *feedback* visual contínuo.

Na camada de entrada, o usuário interage com o ambiente virtual através de gestos capturados pelo rastreamento de mãos do Meta Quest 3 e por meio de interações diretas com objetos da cena. Essas interações são processadas pelo *Storyteller*,

componente central responsável por gerenciar a sequência de execução do procedimento simulado.

O *Storyteller* controla a progressão através de *Stories* (etapas principais do procedimento) e *Steps* (ações atômicas dentro de cada etapa), organizados hierarquicamente. Cada *Story* representa uma fase completa do procedimento, como a higienização das mãos ou a interação com o paciente. Dentro de cada *Story*, os *Steps* individuais encapsulam ações específicas que o usuário deve executar, como selecionar a torneira corretamente, aplicar sabão ou identificar adequadamente o paciente.

O sistema de ancoragem, composto pelo *AnchorsManager* e pelos objetos *Anchor* distribuídos na cena, estabelece a conexão entre a estrutura lógica de progressão (*Stories* e *Steps*) e as posições físicas no ambiente virtual. Cada âncora é identificada por uma *tag* (como "torneira", "dispensador" ou "paciente") e determina onde os marcadores visuais serão posicionados para orientar o usuário durante a execução das etapas.

A validação das ações do usuário ocorre de forma contínua através do sistema de *feedback* visual. Marcadores coloridos (esferas azuis, verdes e vermelhas) indicam respectivamente: etapas disponíveis, ações corretas executadas e erros cometidos. Para etapas que envolvem interação com o paciente, o sistema de diálogo é ativado automaticamente, apresentando caixas de texto estruturadas com perguntas, opções de resposta e justificativas pedagógicas.

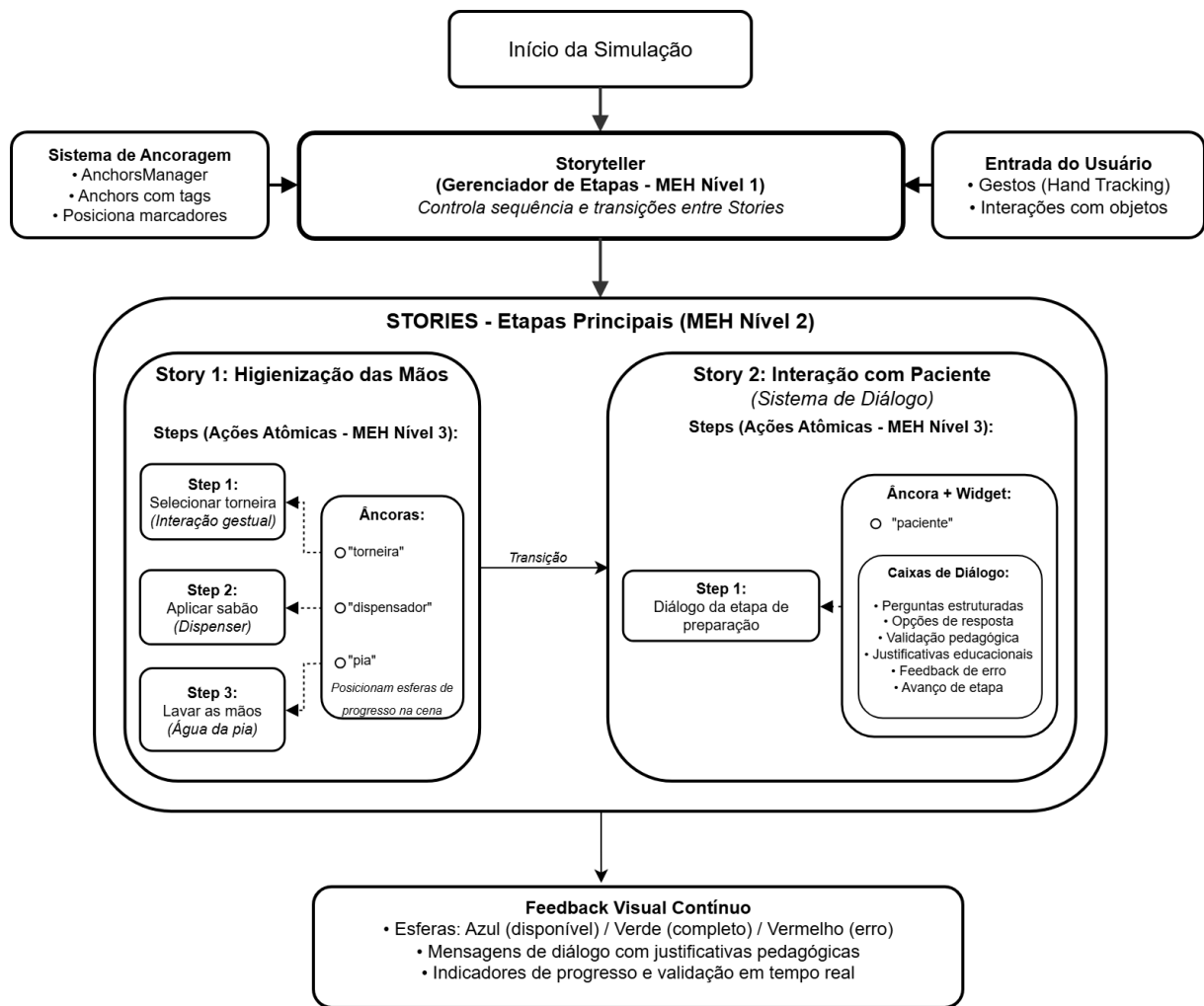
O fluxo completo de uma etapa típica ocorre da seguinte forma: (1) o *Storyteller* instancia uma *Story*, que instancia seus *Steps* associados; (2) marcadores visuais são posicionados nas âncoras correspondentes; (3) o usuário interage com os marcadores ou executa gestos reconhecidos pelo sistema; (4) a validação é realizada e *feedback* apropriado é fornecido; (5) ao completar todos os *Steps* de uma *Story*, o sistema progride automaticamente para a próxima etapa até a conclusão do procedimento.

Essa arquitetura modular garante extensibilidade para adição de novas etapas e facilita ajustes na sequência do procedimento sem necessidade de reconfiguração completa do sistema. Os detalhes de implementação de cada componente são apresentados nas subseções 5.2.7, 5.2.8 e 5.2.9.

5.2.2 Contexto e Impacto da Atualização Tecnológica

O escopo inicial deste trabalho consistia na retomada do desenvolvimento de um projeto que se encontrava estagnado, mas possuía grande potencial de contribuição para as áreas de Computação e Educação em Saúde. O objetivo planejado

Figura 8: Arquitetura funcional do sistema



Fonte: Autoria própria.

era aprimorar a simulação existente com funcionalidades anteriormente não implementadas, completando a sequência de passos do procedimento de punção venosa periférica.

Contudo, fatores não previstos durante o planejamento inicial revelaram a urgência de uma necessidade consideravelmente maior: a modernização tecnológica do projeto VIDA. Essa constatação surgiu a partir da identificação de riscos técnicos significativos que comprometiam não apenas a continuidade do desenvolvimento, mas também a própria viabilidade de execução da versão existente.

No contexto de gerenciamento de projetos, riscos podem ser definidos como eventos ou condições incertas que, caso aconteçam, têm efeitos negativos ou positivos em um ou mais objetivos do projeto. Nesse aspecto, riscos negativos são denominados ameaças e riscos positivos, oportunidades (Project Management Institute,

2021). O *Project Management Body of Knowledge* (PMBOK) estabelece que, ainda que riscos identificados possam ou não se materializar, riscos conhecidos e emergentes são identificados e avaliados ao longo do ciclo de vida de um projeto, a fim de maximizar as oportunidades e minimizar a exposição às ameaças. Ameaças podem impor atrasos no cronograma, perda de reputação, falhas técnicas e queda de desempenho (Project Management Institute, 2021). Portanto, a identificação e o gerenciamento adequado de riscos são fundamentais para o sucesso do projeto VIDA, especialmente porque envolve tecnologias em evolução contínua.

O primeiro risco identificado foi a obsolescência tecnológica do motor gráfico. O projeto VIDA foi desenvolvido na versão 4.19 da *Unreal Engine*, lançada em março de 2018 (GAMES, 2018). Desde então, foram lançadas aproximadamente 18 novas versões do motor gráfico, incluindo atualizações da linha *Unreal 4*, *hotfixes* incrementais da versão 4.19. e, principalmente, a série *Unreal 5*, que introduziu avanços significativos em renderização, iluminação dinâmica (Lumen) e geometria virtualizada (Nanite) (GAMES, 2022).

Essas atualizações incorporaram melhorias substanciais em recursos de RV, compatibilidade com HMDs mais recentes e *plugins* desenvolvidos exclusivamente para versões atualizadas. Por exemplo, o *Meta XR SDK* (META, 2025c), utilizado neste trabalho para rastreamento de mãos, possui funcionalidades avançadas disponíveis apenas a partir da *Unreal Engine* 5.3. Portanto, manter o projeto na versão 4.19. representaria uma limitação crítica ao desenvolvimento, restringindo possibilidades de melhoria em desempenho, qualidade visual e experiência do usuário.

O segundo risco observado foi a fragmentação de versões entre instituições. Durante a fase de recuperação do projeto, identificou-se divergências significativas entre as versões disponíveis no INTERLAB e na EERP-USP. A versão mantida pela EERP-USP estava hospedada em um repositório com arquivos e configurações distintos do projeto existente nos computadores do INTERLAB, além de incluir um executável *standalone* utilizado pelos membros da equipe de enfermagem. A versão do INTERLAB, por sua vez, apresentou diversos desafios críticos de execução e reprodução, conforme detalhado na Subseção 5.2.3.

Essa fragmentação impactava negativamente a retomada do desenvolvimento em múltiplas frentes, pois dificultava o alinhamento técnico entre as equipes, prolongava o tempo necessário para integração de melhorias e exigia verificação manual de diferenças entre versões.

O terceiro risco, por fim, relaciona-se com a obsolescência de *hardware* e dependências tecnológicas. O projeto foi originalmente implementado considerando o uso conjunto do *LeapMotion*, dispositivo de captura de movimentos para rastreamento

de mãos e dedos, e do *OculusRift*, que, HMD da primeira geração de dispositivos de RV imersiva da Meta. À época do desenvolvimento, o uso combinado dessas tecnologias, ilustradas na Figura 9, representava uma solução de tendência na indústria, mesclando vantagens dos dois componentes (SOUZA-JÚNIOR, 2018). 5.2.3.

Figura 9: Dispositivos *LeapMotion* e *Oculus Rift*, utilizados na primeira versão do projeto VIDA Enfermagem.



Fonte: Retirado de Souza-Júnior (2018).

O *OculusRift*, entretanto, foi descontinuado pela Meta nos anos seguintes, sendo substituído pela linha *MetaQuest*, que introduziu rastreamento de mãos nativo através de câmeras integradas do HMD, dispensando dispositivos auxiliares externos. O *LeapMotion*, por sua vez, tornou-se um produto obsoleto sem suporte ativo do fabricante, com *plugins* incompatíveis com versões mais recentes da *Unreal Engine* e sem atualizações para novas plataformas de RV.

Considerando os avanços tecnológicos posteriores à primeira fase de desenvolvimento do projeto VIDA e os recursos de *hardware* disponíveis no *interlab*, exclusivamente da linha *Meta Quest*, impôs-se a necessidade de modernizar o simulador antes de aprimorar funcionalidades existentes. Manter as dependências tecnológicas originais é, portanto, uma ameaça que tornaria o desenvolvimento futuro do projeto inviável, tanto pela indisponibilidade física dos dispositivos quanto pela sua incompatibilidade com tecnologias atuais.

Diante dos riscos identificados, aplicou-se uma estratégia de resposta aos riscos baseada nos princípios do PMBOK (Project Management Institute, 2021), que classifica respostas a ameaças em cinco categorias: evitar, escalar, transferir, miti-

gar ou aceitar. No contexto deste trabalho, optou-se por uma abordagem de *evitar* e *mitigar*, por meio da alteração do escopo original do projeto.

A estratégia de resposta definida envolveu três ações principais. A primeira consistiu na migração da versão do motor gráfico para uma mais recente. A segunda na substituição do *hardware* obsoleto, deprecando dependências do *LeapMotion* e o uso do *Oculus Rift* em favor do *Meta Quest 3* e seus *plugins* de rastreamento. Por fim, a terceira ação consistiu na construção de um repositório centralizado para o projeto com controle de versionamento adequado, alcançando uma versão unificada para uso de ambas as instituições.

Essa mudança de escopo teve como objetivo primário evitar nova estagnação do projeto e garantir sua continuidade em fases mais avançadas de desenvolvimento. Além disso, buscou-se eliminar impedimentos técnicos ao desenvolvimento das etapas do procedimento de punção venosa periférica no simulador, viabilizando novas funcionalidades, execução em múltiplos ambientes e garantindo reprodutibilidade e portabilidade do projeto.

A Subseção 5.2.3 detalha os desafios técnicos encontrados durante a implementação da estratégia de resposta aos riscos. É importante destacar que a identificação de riscos e sua mitigação ocorreram de forma iterativa e concomitante, uma vez que obstáculos adicionais surgiram durante o processo de primeiro contato com o projeto. Por exemplo, a complexidade inesperada da migração entre versões distantes do motor gráfico, a perda de lógicas originais devido à remoção de *plugins* incompatíveis, a necessidade de reconstrução de componentes e o esforço consideravelmente superior ao estimado inicialmente.

Apesar dos desafios, a atualização tecnológica estabelece uma fundação sólida para o desenvolvimento futuro do projeto. Entre os principais impactos positivos, destaca-se a ampliação da capacidade técnica, possibilitando o uso de recursos mais avançados do motor gráfico que proporcionam, por exemplo, maior realismo visual e imersão na simulação. Também há ganhos em portabilidade e reprodutibilidade, pois a versão modernizada executa de forma estável em diferentes computadores e sistemas operacionais. Por fim, a modernização abre caminho para futuras melhorias, porque incorpora suporte ao *Meta Quest 3*, garantindo compatibilidade com tecnologias de RV difundidas e disponíveis no mercado, além de mantidas ativamente pelo fabricante.

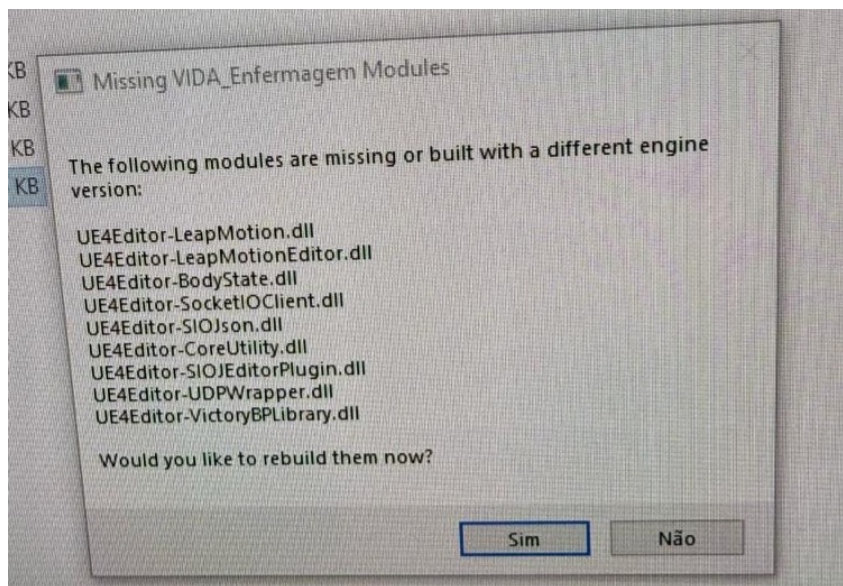
5.2.3 Recuperação e Diagnóstico do Projeto Original

Para o início do desenvolvimento, coordenou-se uma análise aprofundada do estado atual do projeto VIDA Enfermagem disponível nas máquinas do INTERLAB, desenvolvido originalmente na versão 4.19.2 da *Unreal Engine*.

Inicialmente, foi preciso resolver problemas de limitação de armazenamento no computador em que estava configurada a versão anterior do projeto, liberando espaço suficiente para a instalação da versão 4.19.2 da *Unreal Engine*.

Solucionada a questão do armazenamento, as tentativas de execução do projeto resultaram em erros de compilação relacionados aos módulos do *LeapMotion*. A mensagem de erro indicava que os módulos haviam sido compilados em versões diferentes da *Unreal Engine* ou não existiam entre os arquivos do projeto, conforme ilustrado na Figura 10. As tentativas de recompilação não foram bem-sucedidas, indicando incompatibilidades mais profundas.

Figura 10: Mensagem de erro referente à compilação de *plugins* em versão incompatível.



Fonte: Capturado pelos autores.

Após múltiplas tentativas, em alinhamento com o professor orientador, testou-se a remoção completa da pasta de *plugins* do projeto. Essa estratégia permitiu abrir o projeto no *Unreal Editor*, porém diversos *assets* apresentavam corrupção decorrente da remoção, impedindo a compilação. Optou-se pela remoção manual dos componentes associados aos *plugins* problemáticos, o que implicou também na perda da estrutura básica de execução do procedimento simulado.

Restava, então, apenas o ambiente da simulação, sem qualquer lógica funcional do procedimento, uma vez que a maior parte da implementação original estava acoplada ao *LeapMotion* e ao processamento de seus sinais. Essa situação decorreu da obsolescência do *hardware LeapMotion*, que inviabilizou não somente a execução adequada do projeto, mas principalmente a manutenção dos *blueprints* com as lógicas originalmente desenvolvidas.

5.2.4 Migração para Versão Atualizada da Engine

Como solução, optou-se pela migração do projeto para a versão 5.4.4. da *Unreal Engine*, possibilitando a reconstrução das funcionalidades através de recursos atualizados e compatíveis com tecnologias mais recentes. Essa versão específica foi escolhida por oferecer suporte ao *plugin OculusHandTools* (META, 2024), alternativa promissora para substituir o *LeapMotion*, que oferece funcionalidades para rastreamento de mãos baseado em poses e gestos calculados através de coordenadas dos dedos em sistema cartesiano. Além disso, versões atualizadas dos *plugins Meta XR* e *Unreal Engine 5 Platform SDK* oferecem funcionalidades aprimoradas de rastreamento de mãos, indisponíveis na versão original do projeto, mas presentes na 5.4.4.

A *Unreal Engine* oferece um recurso nativo para atualização de versão através da opção *Switch Unreal Engine Version*. Contudo, esse mecanismo apresenta limitações significativas quando aplicado a migrações entre versões muito distantes. As tentativas de migração direta da versão 4.19 para a 5.4.4 resultaram consistentemente em erros de compilação devido à corrupção de arquivos durante o processo de cópia automática.

Diante dessa limitação, adotou-se a transferência manual dos *assets* para um projeto novo e vazio criado na versão 5.4.4, realizada através da manipulação direta dos arquivos no diretório do projeto. Apesar de viabilizar a continuidade do desenvolvimento, essa solução implicou na perda de relações lógicas e referências entre os objetos, exigindo retrabalho manual de reassociação. Por exemplo, as texturas da cena precisaram ser revinculadas individualmente aos elementos através da comparação com a versão antiga. Adicionalmente, recursos interativos, como as partículas da torneira e do dispensador de sabão, demandaram reconstrução completa devido à incompatibilidade com os sistemas da *Engine* mais recente.

Considerando as perdas acumuladas, conduziu-se uma reavaliação dos requisitos e do escopo do projeto, visando o desenvolvimento de uma solução coesa e funcional, ainda que com escopo reduzido em relação ao projeto original.

5.2.5 Sistema de Partículas

Após a importação dos *assets* da versão anterior do projeto, iniciou-se a recuperação inicial da cena de procedimento. Dois objetos fundamentais abordados nesta cena foram a pia e o dispensador de sabão, que deveriam reagir à interação por aproximação do usuário e exibir feixes de partículas simulando água e sabão respectivamente. Entretanto, observou-se que os sistemas de partículas originais do projeto estavam obsoletos devido à descontinuação do sistema *Cascade* pela Epic Games, tornando impossível sua reprodução na Unreal Engine 5.4.4.

Diante dessa incompatibilidade, optou-se por desenvolver dois novos sistemas de partículas utilizando o *Niagara*, sistema de efeitos visuais moderno introduzido na Unreal Engine 4.20 que se tornou o padrão para criação de partículas nas versões mais recentes. O *Niagara* oferece arquitetura baseada em módulos reutilizáveis e editáveis, permitindo controle granular sobre o comportamento das partículas através de um sistema visual de programação baseado em nós. Diferentemente do sistema *Cascade*, o *Niagara* permite a criação de efeitos complexos com melhor desempenho e maior flexibilidade através de seus *emitters* (emissores) e *systems* (sistemas), que podem ser parametrizados e ajustados em tempo real durante o desenvolvimento (??).

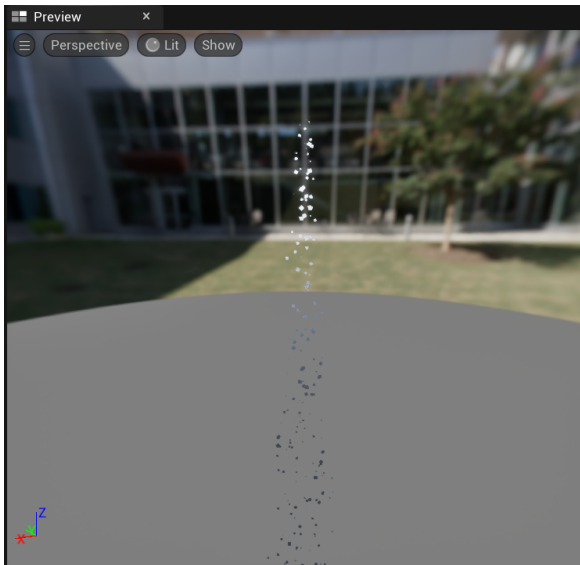
Para o sistema de partículas da água, utilizou-se um material de água importado dos *assets* padrão da Unreal Engine e configurou-se um emissor com formato cônico (*cone shape*) com abertura voltada para baixo. O sistema foi parametrizado para emissão contínua e repetição infinita (*looping*), reproduzindo o efeito visual de uma torneira aberta com fluxo constante de água. Os parâmetros de velocidade inicial (*initial velocity*) e gravidade (*gravity*) foram ajustados para simular o comportamento físico realista do líquido caindo.

Similarmente, para as partículas de sabão líquido, selecionou-se um material translúcido dos materiais padrão da Unreal Engine, gerando-se um feixe cônico com abertura significativamente menor que o da água. Diferentemente do sistema anterior, configurou-se este emissor sem repetição infinita (*non-looping*), de modo que cada acionamento produz um único jato de sabão. Além disso, os parâmetros de velocidade e tempo de vida (*lifetime*) das partículas foram modificados para valores inferiores, gerando a impressão visual de um líquido mais viscoso e denso, característico do sabão líquido.

A Figura 11 ilustra os sistemas de partículas desenvolvidos em funcionamento. Após a implementação e parametrização desses sistemas, foi possível associar novamente a detecção de proximidade do jogador através dos *collision boxes* da pia e do dispensador de sabão à ativação dos respectivos emissores de partículas, restau-

rando a funcionalidade de interação presente na versão anterior do projeto.

Figura 11: Sistemas de partículas Niagara desenvolvidos para água e sabão.



(a) Sistema de partículas da água simulando torneira aberta com fluxo contínuo.



(b) Sistema de partículas do sabão líquido com comportamento viscoso e jato único.

Fonte: Capturado pelos autores.

5.2.6 Interação com gestos

Paralelamente ao processo de migração, dedicou-se esforço ao desenvolvimento de recursos para reconhecimento de gestos que substituíssem o uso do *LeapMotion* na versão mais recente da *Engine*.

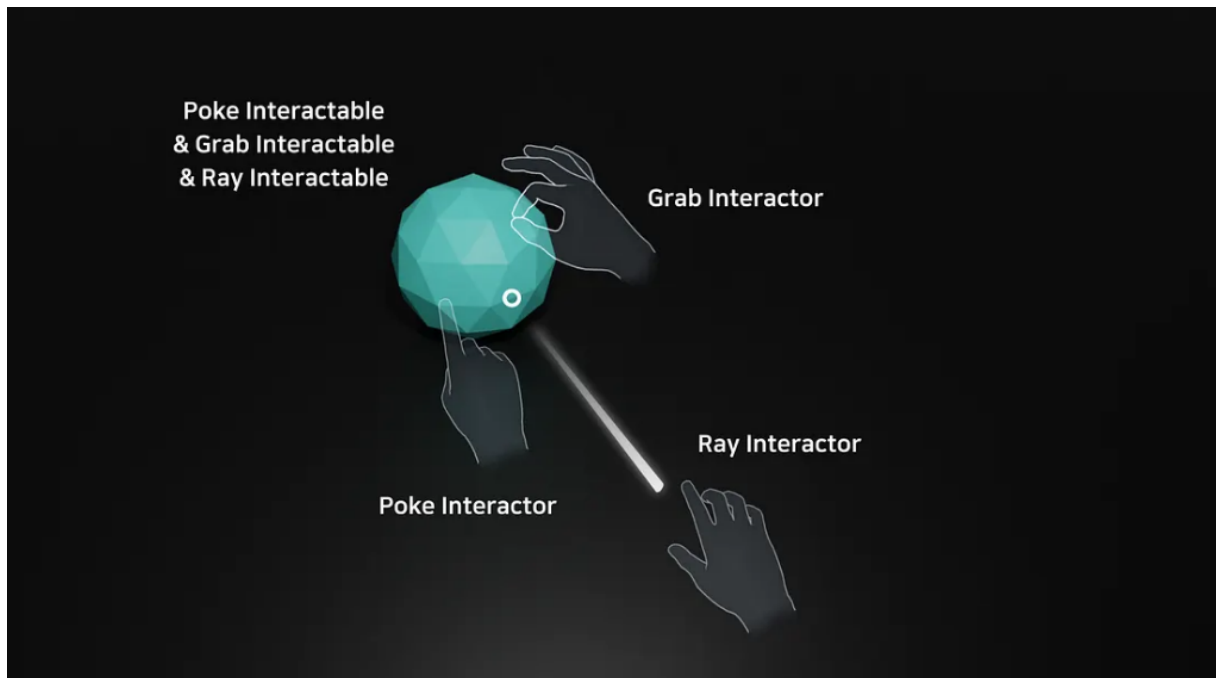
Inicialmente, adotou-se a biblioteca *OculusHandTools* (META, 2024), como mencionado anteriormente, que permite a identificação de gestos específicos realizados com as mãos pelo sistema. Entretanto, essa solução apresentou instabilidades na detecção dos gestos e no acionamento de interações com os objetos da cena. Além disso, a documentação escassa do *plugin* dificultou bastante avanços significativos dessa funcionalidade.

Diante dessas limitações, optou-se pela implementação utilizando o *Meta XR Interaction SDK* (META, 2025c), que oferece documentação e referências de uso mais abrangentes, além de desempenho consideravelmente mais estável no rastreamento e reconhecimento de interações gestuais.

O sistema de interação implementado utiliza dois tipos principais de gestos fornecidos pelo *Interaction SDK* da Meta, conforme ilustrado na Figura 12. O primeiro, *HandGrabInteraction*, permite ao usuário pegar e manipular objetos tridimensionais

na cena, como equipamentos médicos e instrumentos do procedimento, por meio de gestos configuráveis. Essa interação é ativada quando o usuário fecha a mão próximo a um objeto interativo, simulando o gesto natural de agarrar, tal qual uma pinça.

Figura 12: Possíveis formas de interação disponíveis no SDK da Meta: *PokeInteractable* (toque), *GrabInteractable* (pegar) e *RayInteractable* (apontar). O sistema implementado utiliza as interações de toque e pegar.



Fonte: (PARK, 2024).

O segundo tipo, *PokeInteraction*, habilita a interação com elementos de interface bidimensional através de um gesto de toque, similar ao acionamento de botões físicos. Esse tipo de interação é utilizado principalmente no sistema de diálogo para confirmar mensagens de erro e avançar nas caixas de texto apresentadas ao usuário.

A combinação desses dois tipos de interação permite uma experiência natural e intuitiva, na qual gestos cotidianos podem ser mapeados diretamente para ações no ambiente virtual, reduzindo a curva de aprendizado e aumentando a imersão do usuário durante a simulação.

5.2.7 Sistema de Evolução da Simulação

Como parte de um projeto desenvolvido para a disciplina PCS3559 - Tecnologias para Aplicações Interativas, ministrada pelo professor orientador Ricardo Nakamura, foi construída uma funcionalidade adicional ao escopo definido para o trabalho

de conclusão de curso.

A funcionalidade desenvolvida consiste em um sistema de validação das ações executadas pelo usuário durante a simulação. Para viabilizar essa implementação, tornou-se necessária a criação de uma nova interface, a qual possibilitasse que o desenvolvimento das etapas do procedimento siga uma sequência estruturada de passos. Assim, o desempenho do usuário pode ser associado ao progresso em cada passo de forma escalável.

Diante disso, destaca-se a etapa de interação entre enfermeiro e paciente, cujos passos exigem uma interface própria direcionada ao usuário. Nesse sentido, determinou-se que esse momento de interação entre ambos os atores da cena seria realizado através de caixas de texto com elementos de seleção no formato de perguntas e respostas.

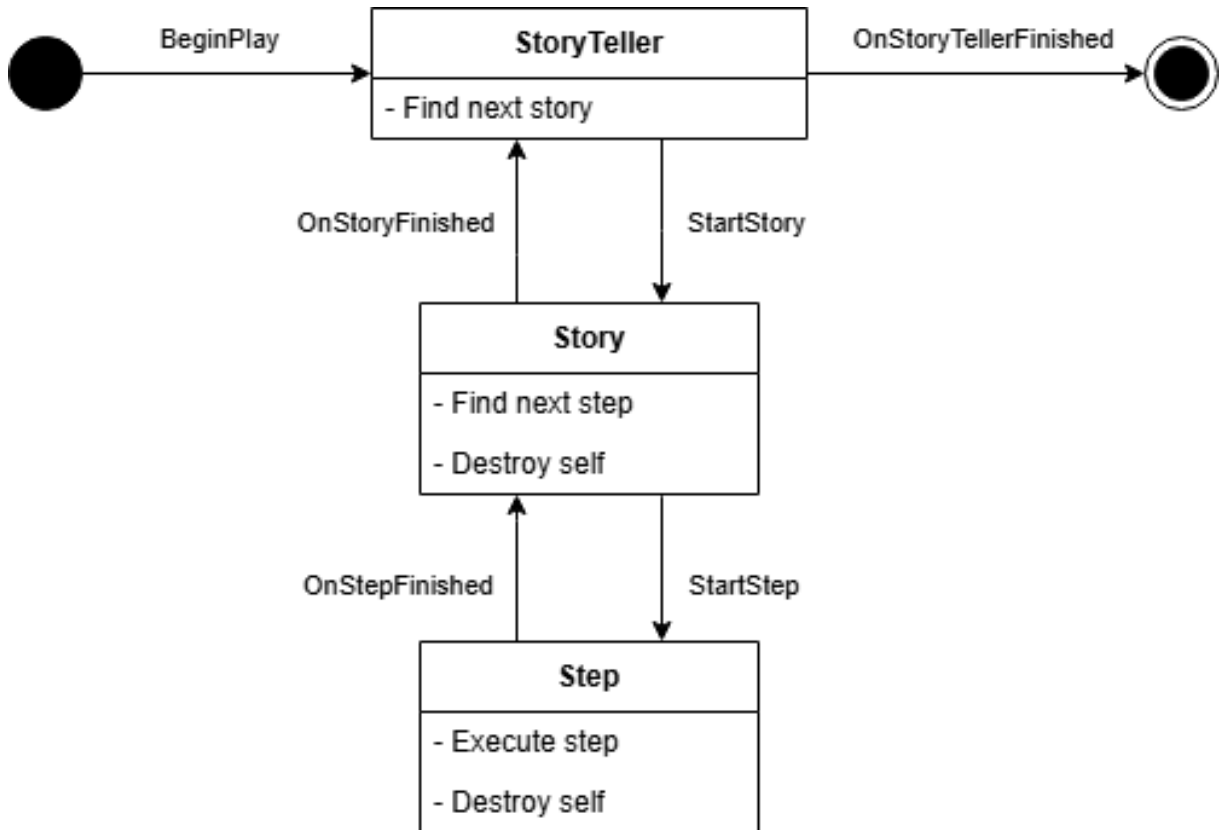
Com isso, estabelece-se uma estrutura primária de diálogo entre os personagens, abrindo caminhos para o aprimoramento técnico desta lógica em etapas mais avançadas do desenvolvimento do projeto VIDA. Por exemplo, seria possível integrar recursos de Inteligência Artificial, como modelos de linguagem natural, e mecanismos de captura de áudio do próprio HMD para a modelagem de uma solução de interação que captura e interpreta a fala do usuário e retorna uma resposta adaptada do paciente. Adicionalmente, esta solução inicial para o diálogo facilita a integração com a lógica desenvolvida para o sistema de evolução da simulação.

A primeira etapa do desenvolvimento do sistema de evolução da simulação consistiu na modelagem da estrutura de progressão de etapas. A arquitetura desenvolvida, baseada na abordagem de decomposição hierárquica proposta por Makrini et al. (2022), utiliza o modelo de MEH, pois se trata de um sistema que demanda controle sequencial estruturado.

Analogamente ao *framework* desenvolvido por Makrini et al. (2022), no qual tarefas de montagem complexas são decompostas recursivamente até ações atômicas, o sistema de evolução da simulação estrutura o gerenciamento do fluxo de progressão do procedimento em três níveis hierárquicos distintos. No nível superior, o *Storyteller* representa a máquina de estados de mais alto nível, gerenciando a sequência inteira do recorte de passos implementado do procedimento e controlando as transições entre as etapas principais. No nível intermediário, as *Stories* correspondem aos estados do *Storyteller*, representando etapas completas da punção, como a higienização das mãos ou a interação com o paciente. Cada *story* é, por si próprio, uma MEH que gerencia seus *steps* internos. Por fim, no nível atômico, os *Steps* encapsulam as ações individuais do usuário, constituindo os estados terminais da hierarquia.

A Figura 13 ilustra o fluxograma completo do algoritmo de progressão do sistema, mostrando a interação entre os três níveis hierárquicos (*Storyteller*, *Stories* e *Steps*) e o ciclo de vida de cada componente desde a inicialização até a conclusão.

Figura 13: Fluxograma do algoritmo de progressão do sistema de evolução.



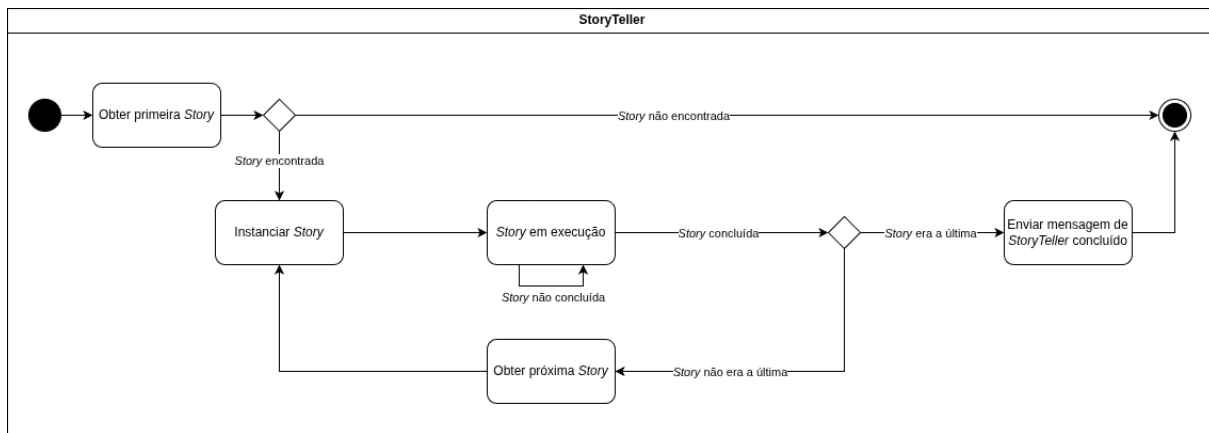
Fonte: Elaborado pelos autores.

A escolha por essa arquitetura hierárquica tem como objetivo facilitar a adição de novas etapas ao procedimento em fases mais avançadas de desenvolvimento do projeto VIDA através da modularidade e proporcionar clareza conceitual por meio da decomposição hierárquica, a qual reflete a natureza sequencial e procedural do procedimento de punção venosa periférica. Dessa forma, é possível, por exemplo, criar uma *story* de higienização das mãos. Nela, os *steps* internos correspondem às ações de o usuário aplicar o sabão em suas mãos, lavá-las e, por fim, secá-las. Em paralelo, há uma *story* de interação com o paciente, a qual abarca a estrutura responsável pelo fluxo de realização das interações de perguntas e respostas.

A Figura 14 ilustra a estrutura geral do sistema de evolução da simulação no nível mais alto. O *Storyteller* representa o módulo de mais alto nível, responsável por gerenciar a sequência de *stories* e suas transições. Ao iniciar a simulação, o sistema ativa a primeira *story* disponível. Quando uma *story* é concluída, o *Storyteller* inicia

a próxima na sequência. O procedimento é finalizado quando não há mais *stories* disponíveis.

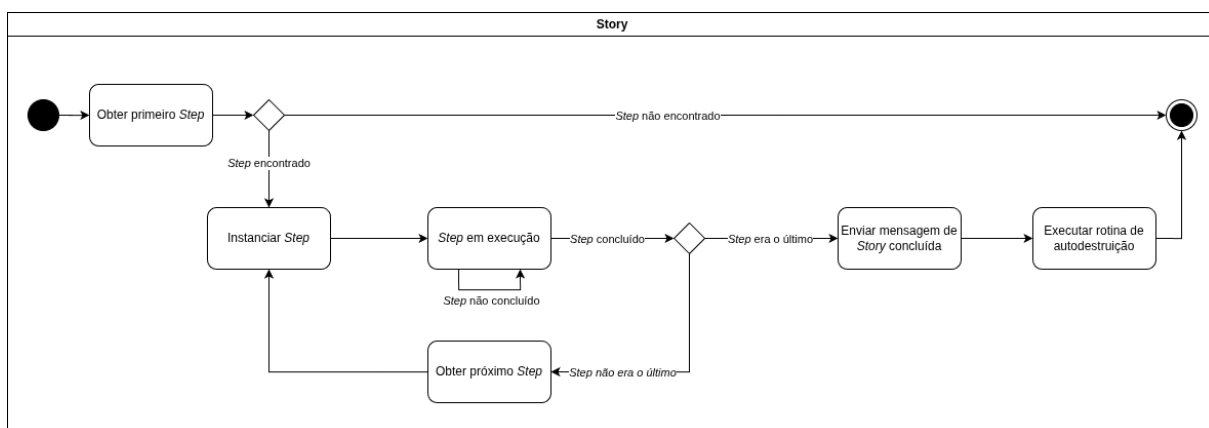
Figura 14: Máquina de estados do *Storyteller*.



Fonte: Elaborado pelos autores.

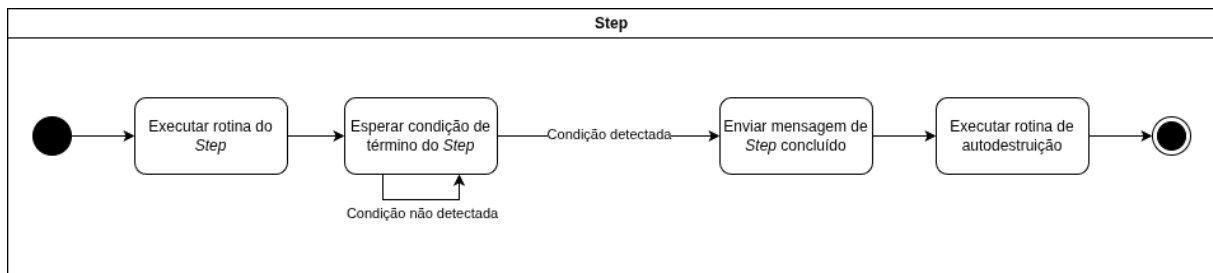
Cada *story*, por sua vez, é composta por uma sequência ordenada de *steps*, conforme ilustrado na Figura 15. A *story* gerencia a instanciamento de seus *steps*, ativando o primeiro ao ser inicializado. Quando um *step* é concluído, ele se autodestrói e notifica a *story*, que então instancia o próximo *step* da sequência. Ao concluir seu último *step*, a *story* envia um sinal de conclusão ao *Storyteller* e se autodestrói.

Figura 15: Máquina de estados de uma *Story*.



Fonte: Elaborado pelos autores.

No nível mais baixo da hierarquia, cada *step* encapsula tanto a ação do usuário quanto sua lógica de validação, como mostra a Figura 16. Ao ser instanciado, o *step* aguarda a ação do usuário. Uma vez que a ação é executada e validada corretamente, o *step* emite um sinal de conclusão e se autodestrói. Essa abordagem permite que a lógica de validação seja personalizada para cada etapa específica do procedimento.

Figura 16: Máquina de estados de um *Step*.

Fonte: Elaborado pelos autores.

Essa estrutura de instanciação e destruição dinâmica garante que, a cada instante, haja apenas um *Storyteller*, uma *Story* e um *Step* ativos na cena, otimizando o uso de recursos computacionais e simplificando o gerenciamento do estado da simulação.

Na solução implementada, utilizam-se referências a classes ao invés de referências a objetos instanciados, permitindo que os componentes sejam criados dinamicamente conforme necessário. Uma instância única do ator da classe *Storyteller* gerencia a criação de instâncias de *stories* em consonância com a sequência definida pelo desenvolvedor. O mesmo vale para cada instância de *story* e de *step*, conforme ilustrado nas Figuras 7, 15 e 16.

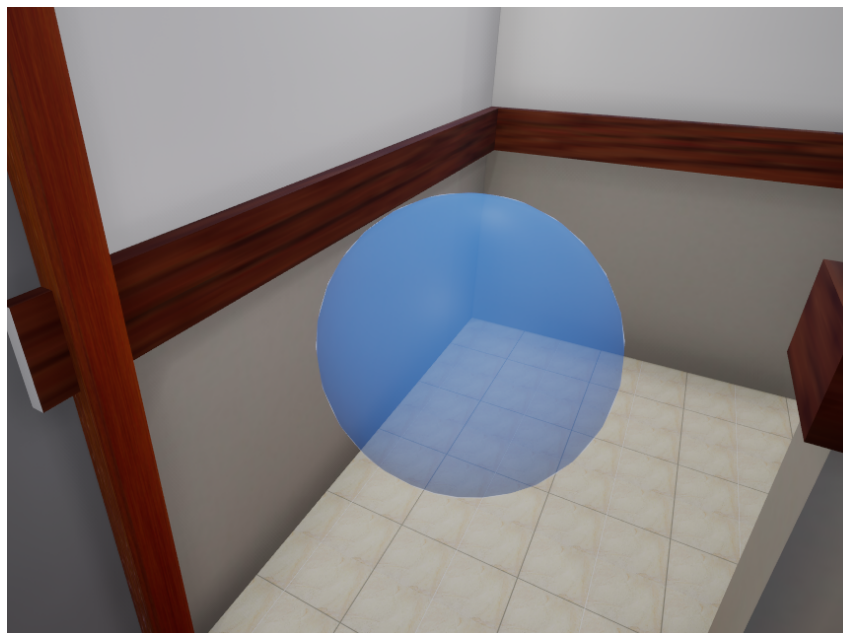
Essa abordagem combina baixo consumo de memória, haja vista que apenas os elementos atualmente ativos estão instanciados, e alta modularidade, permitindo que novas *stories* e *steps* sejam adicionados de maneira facilitada por meio da criação de subclasses. Além disso, essa estratégia facilita a manutenção e expansão do sistema em etapas avançadas de seu desenvolvimento, uma vez que a lógica de cada *step* é encapsulada em sua própria classe, seguindo os princípios de modularidade das MEH.

Uma vez determinada a estrutura de base, prosseguiu-se com o desenvolvimento da interface do sistema de avaliação. Em alinhamento com a professora Simone, definiu-se que seriam exibidos apenas três marcadores na cena, correspondentes aos próximos três *steps* ou *stories*, em posições que sinalizassem a área em que as etapas devem ser executadas.

Os marcadores consistem em esferas azuis selecionáveis pelo usuário, como exibido em 17. Caso a esfera correspondente ao próximo *step* ou *story* seja escolhida, sua cor muda para verde e ela desaparece da cena, dando início à execução do *step* ou *story*. Por outro lado, a esfera permanece vermelha por dois segundos caso não corresponda ao *step* ou *story* seguinte. Nesse caso, exibe-se uma mensagem elucidando o motivo pelo qual a escolha é incorreta, permitindo aprendizado e correção

imediate de erros por parte do usuário.

Figura 17: Instância de esfera azul na cena do procedimento



Fonte: Capturado pelos autores.

A introdução da interface interativa implicou mudanças na estrutura original do sistema de evolução da sequência do procedimento. Anteriormente, os passos eram disparados assim que o gesto correto do usuário os ativasse. Com a nova implementação, os passos são iniciados uma vez que a esfera correspondente seja corretamente selecionada pelo usuário. Portanto, o gerenciamento da posição das esferas tornou-se um ponto crucial para a continuidade do desenvolvimento do sistema.

A posição das esferas foi associada a uma nova classe de objetos: *anchors*. Essa classe pode ser ajustada pelo desenvolvedor nos pontos necessários da cena. Como os *anchors* são instâncias de uma classe aplicadas diretamente à cena, não podem ser introduzidos às definições da classe de *story*, tal como os *steps* são. Isso ocorre porque um objeto deve ser instanciado para manter uma referência a outro objeto instanciado em seus valores padrão. Assim, enquanto os *stories* mantêm referências pré-definidas às classes dos *steps*, *anchors* podem apenas ser associados dinamicamente.

À luz disso, decidiu-se alterar a estrutura base de operação do *storyteller* e adicionar um novo recurso de desenvolvimento: uma interface de gerenciamento do *storyteller*. Essa interface proporciona dois ambientes distintos:

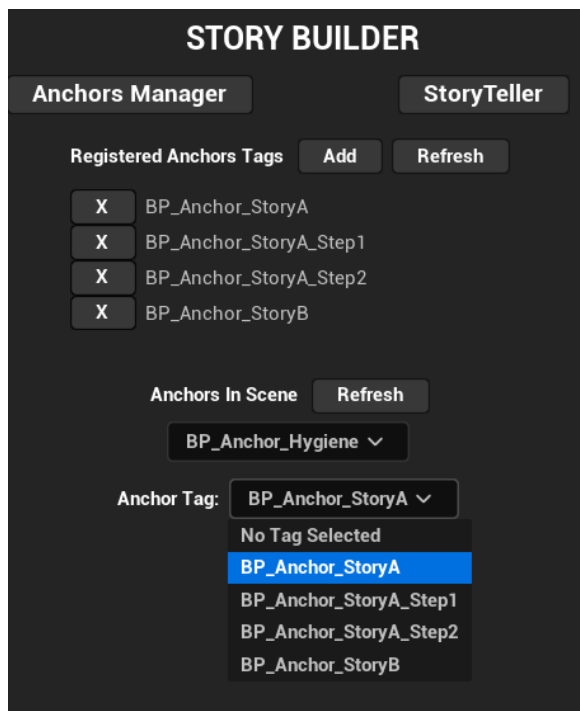
- Ambiente de gerenciamento de *anchors*: o desenvolvedor pode definir uma lista de nomes de *anchors* e selecionar qualquer *anchor* instanciado na cena para

atribuir a ele um nome da lista;

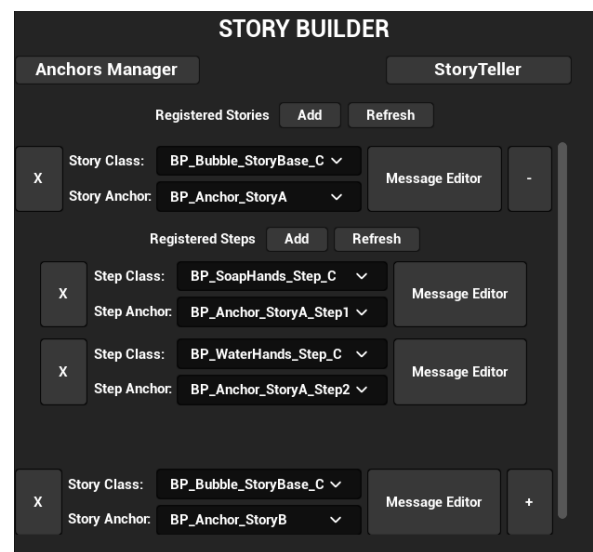
- Ambiente de definição da sequência: O desenvolvedor pode explicitar a estrutura de sequência de passos, indicando as *stories* do *storyteller* e associando a cada uma delas um nome da lista de *anchors*. Além disso, é possível indicar os *steps*, referenciando-os a um nome de *anchor* para cada *story*.

Esses ambientes estão representados nas Figuras 18.

Figura 18: Capturas de tela da interface desenvolvida para o gerenciamento de âncoras e de evolução do procedimento respectivamente.



(a) Gerenciador de Âncoras.



(b) Gerenciador do Storyteller.

Fonte: Capturado pelos autores.

Com essa implementação, a adição de novos elementos na sequência de ações do procedimento é pautada principalmente pela criação de subclasses da classe *step*, tornando o processo de expansão do sistema mais modular e intuitivo.

5.2.8 Sistema de Âncoras

O sistema de âncoras foi desenvolvido para solucionar o desafio de posicionamento dinâmico dos marcadores visuais no ambiente da simulação. Como os marcadores precisam indicar ao usuário onde executar cada etapa do procedimento, tornou-se necessário criar uma solução que permitisse aos desenvolvedores definir posições específicas na cena de forma intuitiva e escalável.

A arquitetura proposta baseia-se em dois componentes principais: o *AnchorManager* e as *Anchor*s (âncoras) propriamente ditas. O *AnchorManager* é uma instância única na cena responsável por centralizar a definição de pontos de referência, enquanto cada *Anchor* representa um ponto específico no espaço tridimensional onde um marcador pode ser posicionado.

O fluxo de configuração do sistema ocorre em duas etapas distintas. Na primeira etapa, o desenvolvedor define uma lista de identificadores textuais no *AnchorManager*, que funcionam como nomes lógicos para os pontos de ancoragem (por exemplo, "pia", "dispensador_sabão", "paciente"). Esses identificadores são posteriormente utilizados como *tags* que podem ser atribuídas aos objetos *Anchor* instanciados na cena. A segunda etapa consiste na atribuição dessas *tags* aos objetos *Anchor* através de uma interface de seleção por *dropdown*, que lista todos os *Anchor*s disponíveis no ambiente.

Quando uma *tag* é atribuída a um *Anchor*, o sistema automaticamente configura o objeto com essa identificação, permitindo que o *Storyteller* localize dinamicamente os pontos de ancoragem durante a execução. O *Storyteller*, ao instanciar uma *story*, busca na cena todos os objetos que possuem a *tag* especificada e utiliza o primeiro objeto encontrado como referência de posicionamento para o marcador visual correspondente. Similarmente, uma *story* também busca na cena os objetos com a *tag* especificada para instanciar um *step*.

Essa abordagem resolve a limitação técnica mencionada anteriormente na Subseção 5.2.7, onde *Anchor*s, por serem objetos instanciados diretamente na cena, não podem ser referenciados estaticamente nas definições de classe de *stories*. A solução por *tags* permite uma associação dinâmica entre as estruturas lógicas (*stories* e *steps*) e as posições físicas no ambiente virtual, mantendo a modularidade do sistema.

A integração entre o sistema de âncoras e o *Storyteller* foi projetada para facilitar a expansão futura do projeto. A interface desenvolvida permite ao programador visualizar simultaneamente a estrutura hierárquica do procedimento e os pontos de ancoragem disponíveis, configurando as associações de forma visual e intuitiva. Cada *story* possui um campo de âncora associado, e dentro de cada *story*, os *steps* individuais também podem ter suas próprias âncoras específicas, permitindo controle granular sobre o posicionamento dos marcadores em diferentes níveis da hierarquia.

Essa arquitetura garante que o sistema de evolução da simulação permaneça independente da disposição física dos objetos na cena, facilitando ajustes no *layout* do ambiente sem necessidade de reconfiguração da lógica do procedimento. Adicionalmente, a abordagem por *tags* permite que múltiplos *steps* ou *stories* compartilhem o mesmo ponto de ancoragem quando apropriado, reduzindo redundância na configu-

ração.

5.2.9 Sistema de Diálogo

O sistema de diálogo foi desenvolvido para estruturar a interação entre o estudante e o paciente virtual durante etapas do procedimento de PVP que requerem comunicação verbal. A implementação integra-se aos sistemas de evolução de simulação e de ancoragem, sendo ativada automaticamente quando *Stories* ou *Steps* relacionados à interação com o paciente são selecionados corretamente pelo usuário.

A arquitetura do sistema baseia-se em *widgets* bidimensionais desenvolvidos nativamente no *Unreal Engine* 5.4.4, encapsulados por um objeto *InteractableWidget* do *plugin* de interações da Meta (META, 2025c). Esse objeto é responsável por tornar a composição de *widgets* que definem um diálogo interagível com o sistema do *Po-keInteraction*. Isso permite, por exemplo, tornar clicáveis os botões que representam cada uma das opções das respostas do diálogo.

Quando uma etapa de diálogo é iniciada, o sistema exibe uma caixa de texto estruturada contendo uma pergunta relacionada ao procedimento e múltiplas opções de resposta. A Figura 19 ilustra um exemplo de *widget* de diálogo durante uma interação de identificação do paciente.

O desenvolvimento do conteúdo pedagógico dos diálogos baseia-se em um modelo de validação colaborativo elaborado em parceria com a EERP-USP, que mapeia perguntas, respostas esperadas e justificativas fundamentadas em protocolos de enfermagem e diretrizes de segurança do paciente. Cada interação de diálogo é estruturada seguindo o formato de perguntas e respostas, onde o estudante deve selecionar a opção mais adequada dentre as alternativas apresentadas.

A Tabela 5.1 exemplifica a estrutura completa de validação pedagógica desenvolvida para a etapa de identificação do paciente, uma das interações fundamentais do procedimento.

Tabela 5.1: Estrutura de validação pedagógica para identificação do paciente.

Campo	Conteúdo
Etapa	Conversa com paciente antes do procedimento.
Contexto	Confirmar o nome do paciente.

Continua na próxima página

Tabela 5.1 – Continuação

Campo	Conteúdo
Pergunta	Escolha a opção mais adequada para confirmar o nome do paciente:"
Opções de Resposta	A) "Qual é o seu nome completo e sua data de nascimento?" (<i>Correta</i>)
	B) "O senhor é o Fulano?" (<i>Incorreta</i>)
	C) "Olhar pulseira de identificação." (<i>Incorreta</i>)
	D) "Qual o seu nome?" (<i>Incorreta</i>)
Justificativa (A)	Essa é a forma padronizada de identificação, utilizando pelo menos dois identificadores (nome completo e data de nascimento), conforme protocolos de segurança do paciente.
Justificativa (B)	A pergunta sugestiva aumenta o risco de erro, pois o paciente pode apenas confirmar sem realmente prestar atenção, gerando risco de identificar o paciente incorretamente.
Justificativa (C)	A pulseira de identificação pode ser extraviada ou podem haver dois pacientes com o mesmo nome. O protocolo padrão é utilizar pelo menos dois identificadores. Caso o paciente não possa responder, a identificação deve ser confirmada com o acompanhante além da pulseira.
Justificativa (D)	Embora melhor que a pergunta sugestiva, não atende ao protocolo de dois identificadores obrigatórios.
Importância	A identificação correta do paciente é etapa crítica para evitar erros de procedimento, garantindo que a punção venosa seja realizada na pessoa certa.

Continua na próxima página

Tabela 5.1 – Continuação

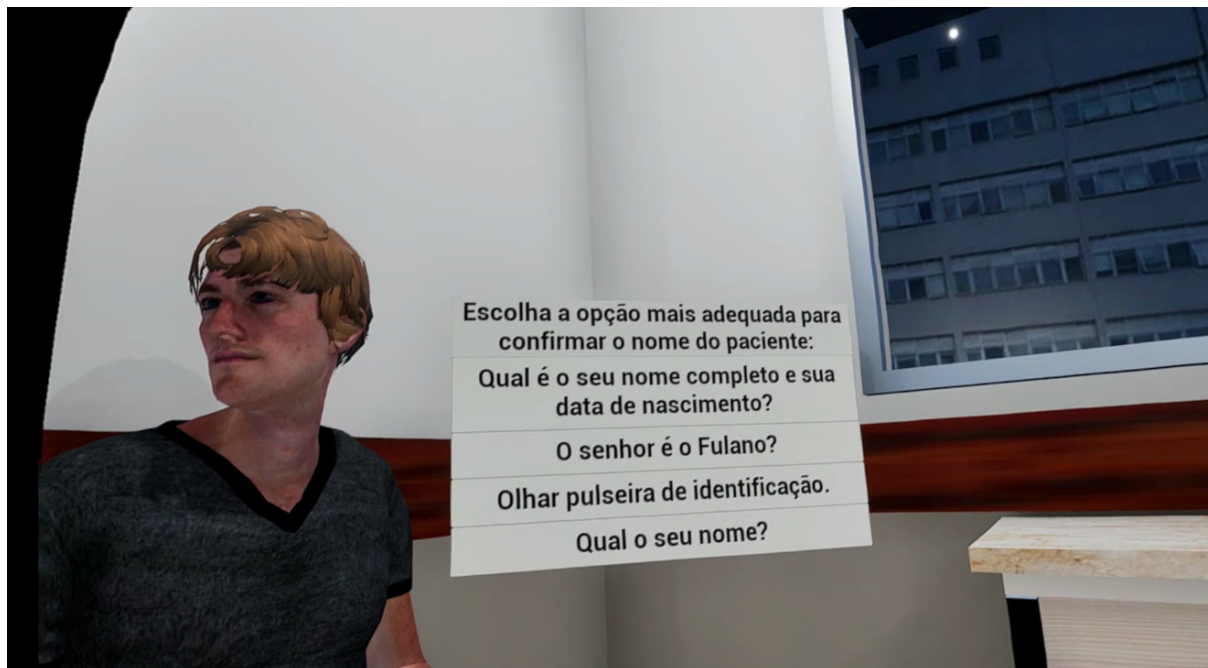
Campo	Conteúdo
Risco Associado	Se o profissional identificar incorretamente o paciente, pode realizar o procedimento em pessoa errada, causando riscos de segurança, atrasos no tratamento e falhas éticas e legais.
Referência	Ministério da Saúde (2013); Conselho Federal de Enfermagem (2017); BRASIL. Agência Nacional de Vigilância Sanitária (2013).
Observações Adicionais	• Sempre utilizar dois identificadores (nome completo e data de nascimento); • Evitar perguntas sugestivas; • Caso o paciente não consiga responder, a identificação deve ser confirmada com acompanhante ou responsável legal, além da pulseira.

A estrutura implementada permite duas modalidades de interação. Na primeira modalidade, o sistema apresenta uma pergunta sobre o procedimento (por exemplo, "Qual a forma correta de identificar o paciente?") e o estudante escolhe entre as opções disponíveis. Na segunda modalidade, potencialmente expansível em desenvolvimentos futuros, o sistema pode exibir uma fala do paciente e solicitar que o estudante escolha a resposta mais apropriada.

O mecanismo de validação opera de forma análoga ao sistema de evolução de simulação: quando o usuário seleciona a resposta correta, o usuário receberá um *feedback* visual positivo, como exposto na Figura 20a, e o *Step* de diálogo é concluído com sucesso e a simulação progride para a próxima etapa. Caso uma resposta incorreta seja escolhida, exibido na Figura 20b, o sistema exibe uma mensagem de erro pedagógica que explica o motivo da incorreção e orienta o estudante sobre a conduta adequada, promovendo aprendizado reflexivo e correção imediata.

Assim como os *widgets* construídos para o sistema de diálogo, o *widget* referente à mensagem de erro foi construído dentro do *Unreal Engine* 5.4.4 e encapsulado por um objeto *InteractableWidget*, viabilizando a realização de um gesto para fechar a caixa de texto exibida por meio de um clique no canto superior direito. A Figura

Figura 19: Exemplo de *widget* de diálogo apresentado ao usuário durante etapa de identificação do paciente.



Fonte: Capturado pelos autores.

21 apresenta o modelo desenvolvido para a mensagem, destacando os elementos de interface como caixas de texto, botões de seleção e áreas de *feedback*.

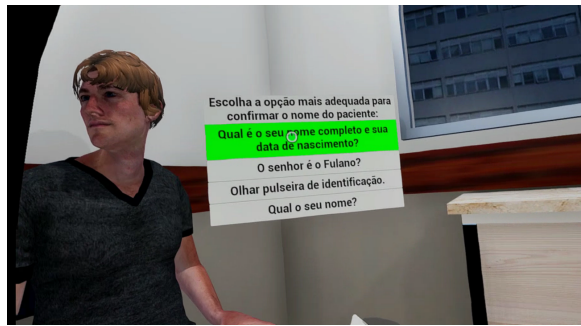
Essa estrutura de validação pedagógica garante que o simulador não apenas reproduza procedimentos técnicos, mas também promova o desenvolvimento de competências críticas relacionadas ao raciocínio clínico e à tomada de decisão, permitindo que o estudante revise a sequência e os detalhes das etapas durante a simulação. A documentação completa do mapeamento de todas as etapas desenvolvidas encontra-se no Apêndice 6.3.

5.2.10 Recuperação do *Asset* do Paciente na Cena do Procedimento

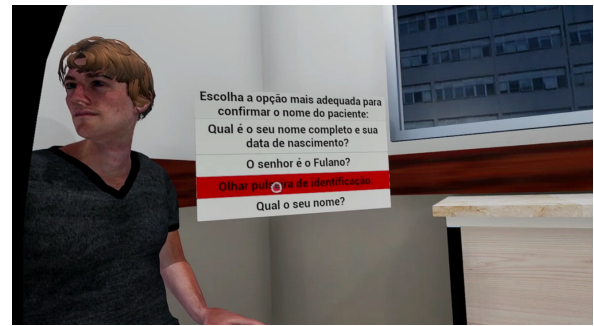
As malhas estática e esquelética, assim como as animações do paciente, foram obtidas originalmente a partir de personagens e animações do Mixamo (ADOBE, 2025). Em função da corrupção dos arquivos originais, optou-se por recuperar as animações diretamente do protótipo anterior desenvolvido em Unreal Engine 4.19.2 (SOUZA-JÚNIOR et al., 2020).

As animações de *Sitting* e *Sitted*, ilustradas pela Figura 22, foram exportadas em formato *.fbx* do projeto original, sem as malhas esqueléticas e contendo apenas

Figura 20: Capturas dos feedbacks visuais de escolhas de opção de diálogo.



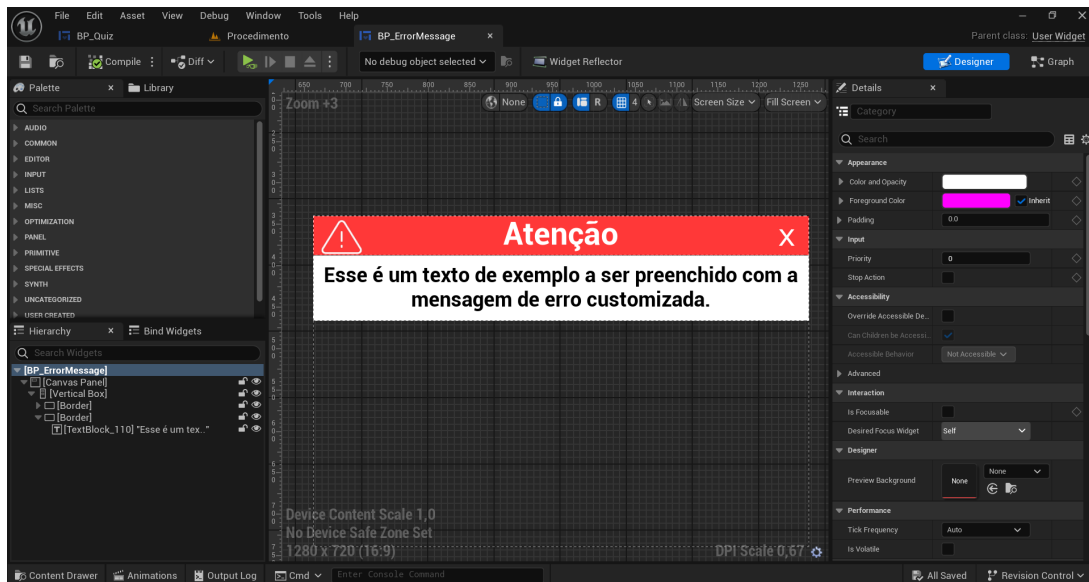
(a) Escolha da opção correta.



(b) Escolha da opção incorreta.

Fonte: Capturado pelos autores.

Figura 21: Modelo do *widget* de erro desenvolvido no *Unreal Engine*.



Fonte: Capturado pelos autores.

os dados de animação, e importadas para o Unreal Engine 5.4.4. Durante os testes de integração, identificou-se um desalinhamento entre o paciente e a cadeira na transição para a animação *Sitted*. Para corrigir esse problema, aplicou-se uma translação de -66 unidades no eixo X diretamente na animação, garantindo o posicionamento correto do personagem sobre a cadeira.

O sistema de animação utiliza a máquina de estados original do protótipo, ilustrada na Figura 23, que prevê múltiplos estados para diferentes etapas do procedimento. No escopo atual do projeto, apenas dois estados foram ativados (*Sitting* e *Sitted*). O paciente inicia no estado *Sitting*, executa a transição para *Sitted* e permanece nesse estado durante toda a simulação. A manutenção da estrutura completa da máquina de estados visa facilitar a expansão futura do sistema com as demais

Figura 22: Animações de *Sitting* e *Sitting Idle* utilizadas do Mixamo no *Unreal Engine Animation Editor*.



(a) *Sitting*

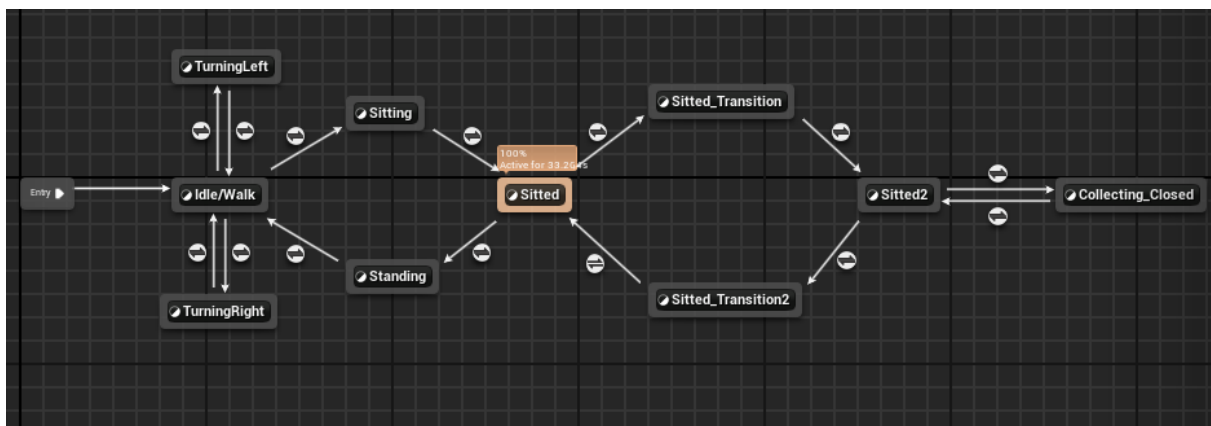


(b) *Sitting Idle*

Fonte: Capturado pelos autores.

animações do procedimento, uma vez que as variáveis de controle para as transições subsequentes não foram implementadas nesta versão. Essa implementação mostrou-se suficiente para os propósitos do projeto, fornecendo o apoio visual necessário aos sistemas de evolução e diálogo.

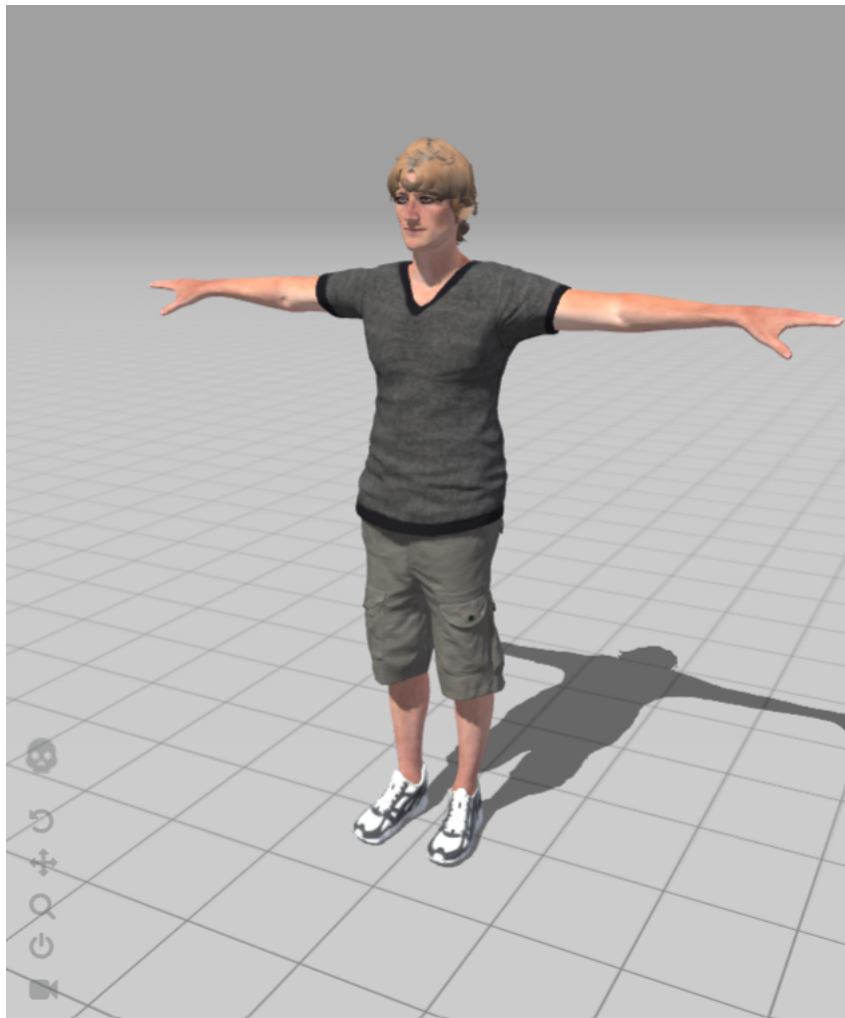
Figura 23: Máquina de Estados implementada para o movimento do paciente na simulação, com foco para a permanência no estado de *Sitted*.



Fonte: Capturado pelos autores.

As Figuras 24 e 25 ilustram, respectivamente, o personagem Remy utilizado para o paciente e sua apresentação final na cena do procedimento.

Figura 24: Personagem *Remy* reutilizado para o personagem paciente.



Fonte: (ADOBE, 2025).

5.2.11 Testes e avaliação

O processo de avaliação do projeto foi estruturado em duas etapas complementares. A primeira etapa consistiu em testes internos, abrangendo a avaliação de desempenho de versões intermediárias e da versão final do aplicativo em um dispositivo Meta Quest 3, além de testes funcionais conduzidos de forma iterativa ao longo de todo o ciclo de desenvolvimento. A segunda etapa envolveu testes de usabilidade com potenciais usuários finais, realizados durante a fase de conclusão do projeto.

5.2.11.1 Testes Internos

A validação interna do sistema concentra-se em dois aspectos principais: a verificação funcional de todos os componentes desenvolvidos e a análise de sua performance operacional. Dedicar-se atenção especial à avaliação do desempenho técnico, incluindo métricas de responsividade da interface, consumo de bateria e qualidade de

Figura 25: Personagem apresentado na cena do procedimento.



Fonte: Capturado pelos autores.

renderização gráfica. Estas análises seguem os padrões técnicos estabelecidos pela Meta, visando proporcionar aos usuários uma experiência de realidade virtual sem interrupções e plenamente imersiva.

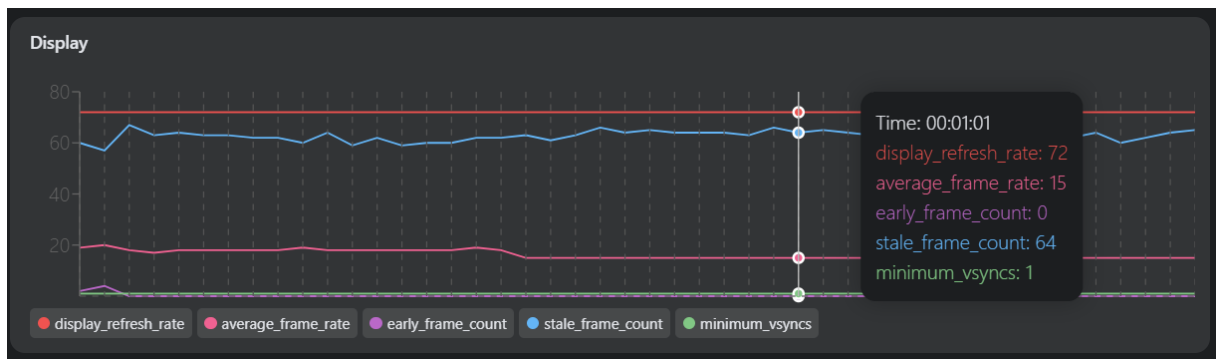
5.2.11.1.1 Avaliação de desempenho no *headset*

Para a avaliação de desempenho do aplicativo, selecionou-se um conjunto de métricas críticas de desempenho a serem monitoradas, utilizando as ferramentas *OVRMetrics Tool* (META, 2025f) e *Performance Analyzer* (META, 2025d) do programa *Meta Quest Developer Hub*. Foram conduzidas 10 sessões de aproximadamente 2 a 3 minutos de uso contínuo da aplicação, desde sua etapa de inicialização. Para fins de análise, consideraram-se os melhores gráficos obtidos dentre as 10 amostras para cada uma das métricas nos cenários avaliados, representando o desempenho ótimo alcançável pela aplicação.

As seguintes métricas foram selecionadas por sua relevância para aplicações de RV:

- *Average FPS (Frames Per Second)*: Indica a taxa de quadros renderizados por segundo. A Meta estabelece que essa métrica deve refletir o valor da *refresh rate* do visor. Para o HMD Meta Quest 3, especificamente, esse valor nunca deve ser inferior a 72 fps, sendo este o limiar mínimo para garantir uma experiência imersiva sem desconforto visual (META, 2025b);
- *CPU Utilization* e *GPU Utilization*: Indicam a porcentagem de utilização da CPU e GPU, respectivamente. No caso da CPU, registra-se o valor do núcleo de

Figura 26: Desempenho de display da simulação com paciente



Fonte: Autoral.

pior desempenho em contextos *multithread*, representando o gargalo potencial do processamento. Para a GPU, componente tipicamente crítico em aplicações de RV devido à complexidade da renderização estereoscópica, o valor não deve ultrapassar 90%, pois acima desse patamar observa-se degradação perceptível da performance (META, 2025f);

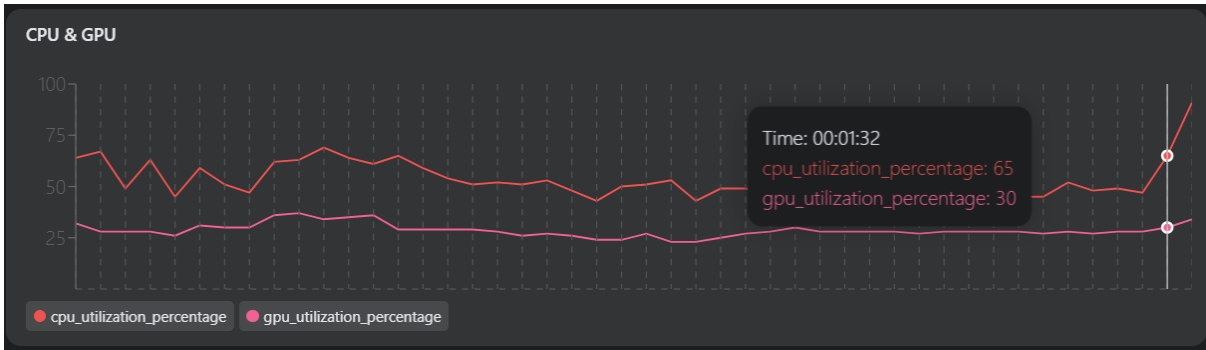
- **APP GPU Time:** Indica o tempo que a aplicação consome para renderizar um único quadro. Esta métrica é fundamental para identificar gargalos de performance, pois permite diagnosticar a origem das limitações. A análise conjunta com o FPS permite determinar: (i) se GPU Time < 13,8ms e FPS < 72, há gargalo na GPU, provavelmente causado por complexidade gráfica excessiva, geometria densa, materiais computacionalmente custosos ou iluminação elaborada; (ii) se GPU Time > 13,8ms e FPS < 72, o gargalo está na CPU, sugerindo problemas na lógica da simulação, como *Blueprints* não otimizados, eventos disparados excessivamente ou processamento intensivo de física (META, 2025e).

Inicialmente, realizaram-se testes em uma cena completa incluindo o modelo do paciente. Os resultados revelaram problemas significativos de desempenho: em todos os casos de teste, o *Average FPS* não alcançou 30 frames por segundo (Figura 26), valor substancialmente inferior ao mínimo recomendado de 72 fps. Essa taxa de quadros comprometida representa apenas 41,7% do desempenho esperado, resultando em uma experiência visual instável e potencialmente causadora de desconforto.

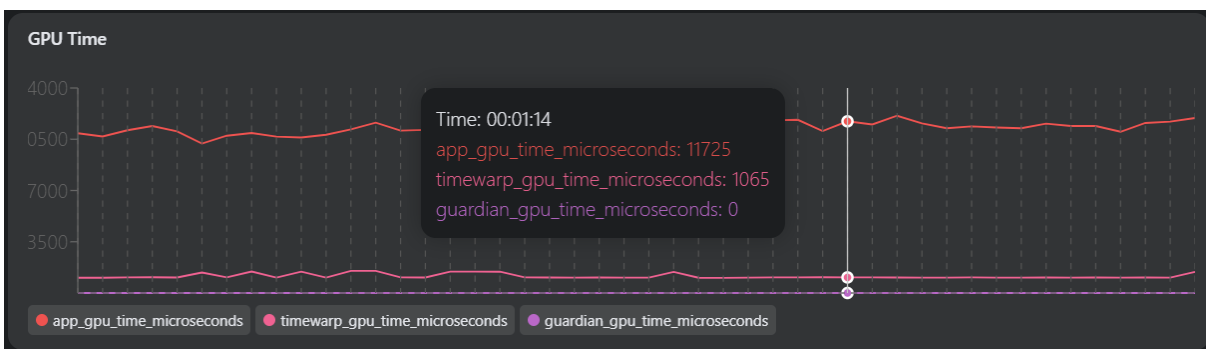
A análise das métricas de utilização de recursos (Figura 27a) revelou um padrão diagnóstico importante: o *CPU utilization* manteve-se consistentemente superior ao *GPU utilization*, enquanto o *GPU Time* permaneceu majoritariamente abaixo do limiar crítico de 13,8ms (Figura 27b). Este perfil de desempenho caracteriza certamente um gargalo de CPU, indicando que o componente limitante não era a capacidade de

renderização gráfica, mas sim o processamento de lógica e física da simulação com o paciente em cena.

Figura 27: Desempenho de GPU e CPU da simulação com paciente



(a) GPU e CPU utilization.



(b) GPU Time

Fonte: Autoral.

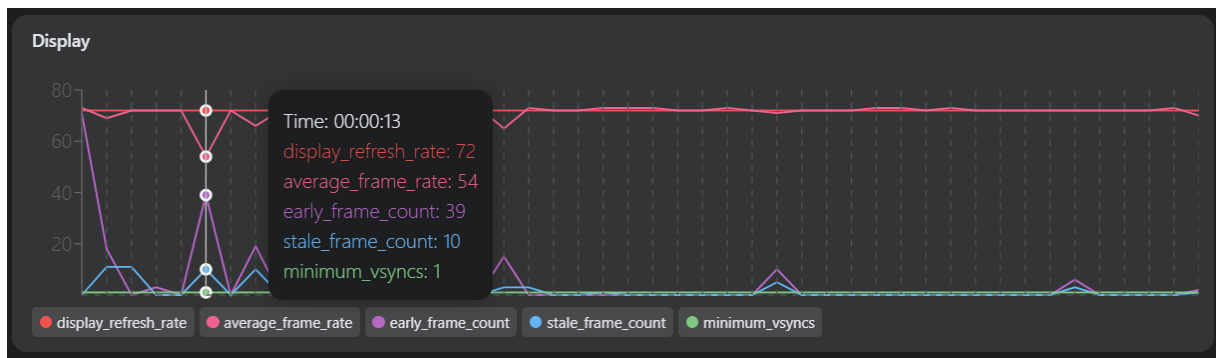
Com base nesta análise, identificou-se o modelo do paciente como provável fonte do gargalo computacional. Elementos complexos em modelos 3D podem impor sobrecarga significativa através de diversos mecanismos: sistemas de animação procedural, detecção de colisões com geometria densa, ou *scripts* de comportamento executados a cada quadro. Para validar esta hipótese, removeu-se o paciente da cena e conduziram-se novos testes.

A remoção do modelo do paciente resultou em melhoria substancial do desempenho em todas as métricas avaliadas. O *Average FPS* apresentou elevação significativa (Figura 28a), embora ainda exibisse quedas pontuais correlacionadas temporalmente com eventos específicos: o início da emissão de partículas da torneira e a ativação do sistema de partículas do sabão. Este comportamento sugere que os sistemas de partículas, embora otimizados, ainda representam picos de demanda computacional durante sua inicialização, possivelmente devido à alocação de recursos e configuração inicial de emissores.

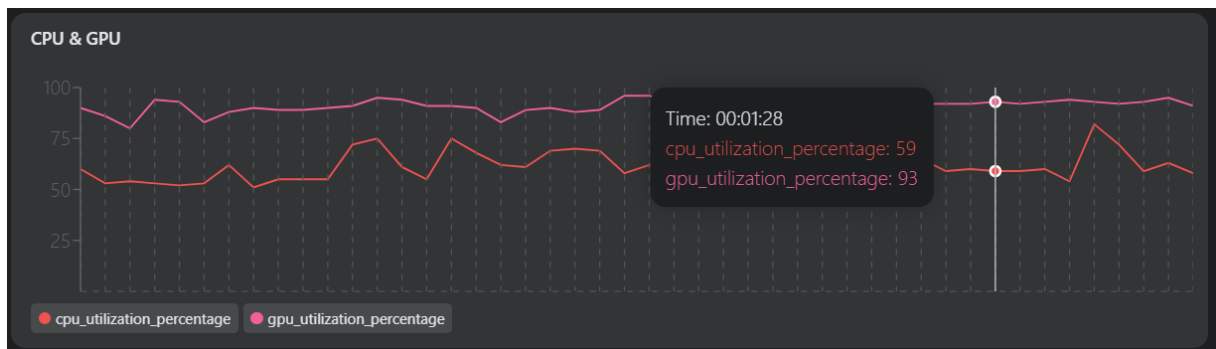
Além disso, observou-se uma inversão na relação entre *CPU utilization* e *GPU utilization* (Figura 28b). Nesse contexto, a GPU passou a apresentar maior utilização que a CPU, configuração esta mais apropriada para aplicações de RV, onde a renderização gráfica naturalmente demanda mais recursos. Esta inversão confirma a resolução do gargalo de CPU anteriormente identificado, redistribuindo a carga computacional de forma mais equilibrada entre os componentes do sistema.

Um aspecto positivo observado em ambos os cenários de teste foi a manutenção constante do *refresh rate* em 72Hz, indicando que o sistema de sincronização vertical (*VSync*) e o mecanismo de *reprojection* do Quest 3 operaram adequadamente, prevenindo artefatos visuais mesmo durante períodos de queda de FPS.

Figura 28: Desempenho da simulação sem o paciente



(a) Desempenho de *display*



(b) Desempenho de GPU e CPU

Fonte: Autoral.

Os testes evidenciaram que o modelo do paciente constituía o principal gargalo de desempenho da aplicação, impondo sobrecarga significativa à CPU que comprometia a taxa de quadros abaixo de níveis aceitáveis. Um fator adicional que pode contribuir para este comportamento é a origem do modelo: trata-se de um personagem antigo proveniente do Mixamo, utilizado originalmente na versão anterior do projeto e reimportado para a versão atual. Modelos e animações provenientes de ver-

sões antigas podem carregar estruturas de *rig*, sistemas de *skinning*, *physics assets* e *animation blueprints* que não se beneficiam das otimizações introduzidas nas versões mais recentes do motor, além de eventualmente conterem dados legados incompatíveis com o fluxo de processamento moderno. Essa defasagem técnica oferece uma explicação consistente para a sobrecarga observada na CPU e para o impacto negativo no desempenho geral da aplicação. Sua remoção da cena resultou em melhoria substancial do desempenho observado, embora revele a necessidade de otimização adicional dos sistemas de partículas para suavizar picos de demanda durante eventos de inicialização.

5.2.11.1.2 Testes Funcionais

Durante todo o ciclo de desenvolvimento, implementou-se uma estratégia de validação contínua das funcionalidades. Cada novo recurso adicionado ao sistema passou por verificação, não apenas isoladamente, mas também considerando seu impacto nas funcionalidades previamente desenvolvidas. Essa abordagem iterativa permitiu detectar e corrigir incompatibilidades antes que se acumulassem.

Os casos de teste apresentados a seguir foram organizados por sistema funcional e associados aos requisitos especificados no Capítulo 4. Cabe destacar que os requisitos MQ3 (Compatibilidade com Meta Quest 3), UNI (Uniformização do protótipo) e GES (Rastreamento das mãos) permeiam todos os testes realizados, sendo validados transversalmente em cada categoria.

Validação de Interações Gestuais

A validação das interações gestuais concentrou-se em verificar a precisão do rastreamento de mãos e a responsividade do sistema aos comandos do usuário.

- Teste 1: Executar movimento de pinça próximo a equipamentos médicos dispostos na cena. O sistema deve capturar o objeto, permitir sua manipulação espacial completa (translação e rotação) e mantê-lo na última posição ao liberar o gesto. Requisitos validados: GES;
- Teste 2: Posicionar a mão nas proximidades da pia. Verificar se o fluxo de partículas de água inicia automaticamente enquanto a mão permanece na região de detecção. Requisitos validados: GES;
- Teste 3: Acionar o dispensador através de aproximação manual. Confirmar que o sistema emite uma única dose de sabão com propriedades viscosas apropriadas, cessando após o jato inicial. Requisitos validados: GES;
- Teste 4: Interagir com botões e elementos de interface através de toque com o dedo indicador. O sistema deve registrar a ação imediatamente, fornecendo

resposta visual instantânea. Requisitos validados: GES, DLG, FBV.

Validação do Fluxo de Progressão

Os testes deste módulo focaram na correta gestão da sequência procedural e na validação das escolhas do usuário.

- Teste 5: Ao inicializar a aplicação, confirmar que três marcadores esféricos azuis aparecem nos locais predefinidos, correspondentes às três próximas ações disponíveis. Requisitos validados: EVS, ANC;
- Teste 6: Interagir com o marcador correspondente à ação imediata. Validar que sua coloração muda para verde, o marcador se dissolve da cena e a etapa associada é ativada. Requisitos validados: EVS, FBV
- Teste 7: Tentar interagir com marcadores fora da sequência estabelecida. O sistema deve manter o marcador em vermelho temporariamente e apresentar orientação textual explicando a inadequação da escolha. Requisitos validados: EVS, FBV;
- Teste 8: Concluir todas as ações de uma Story específica. Verificar se o sistema automaticamente inicia a Story seguinte, reorganizando os marcadores conforme as novas âncoras definidas. Requisitos validados: EVS, ANC;
- Teste 9: Executar o ciclo completo das Stories implementadas. Confirmar que o Storyteller encerra adequadamente ao esgotar a sequência programada. Requisitos validados: EVS.

Validação do Sistema de Ancoragem

Estes testes avaliaram a flexibilidade e robustez do sistema de posicionamento dinâmico.

- Teste 10: Através da interface de gerenciamento, trocar a referência de um Anchor na cena. Durante a execução, verificar se os marcadores aparecem na localização atualizada. Requisitos validados: ANC;
- Teste 11: Configurar múltiplos Anchors com identificadores distintos. Validar que o sistema recupera corretamente cada Anchor baseado em sua tag durante a progressão. Requisitos validados: ANC, EVS;
- Teste 12: Associar a mesma tag de Anchor a diversos Steps sequenciais. Confirmar que os marcadores mantêm posicionamento consistente ao longo da Story. Requisitos validados: ANC.

Validação da Estrutura de Diálogo

Os testes deste sistema focaram tanto na funcionalidade técnica quanto na adequação pedagógica do conteúdo do diálogo.

- Teste 13: Ativar um Step de diálogo. Verificar se o widget apresenta claramente a pergunta e todas as alternativas, permitindo seleção através de interação por toque. Requisitos validados: DLG, GES;
- Teste 14: Escolher a alternativa pedagogicamente correta. O sistema deve sinalizar visualmente o acerto (coloração verde) e avançar automaticamente para o Step subsequente. Requisitos validados: DLG, FBV, VPD;
- Teste 15: Selecionar uma alternativa inadequada. Validar que o sistema exibe sinalização visual de erro (coloração vermelha) acompanhada de texto explicativo fundamentado em protocolos clínicos. Requisitos validados: DLG, FBV, VPD;
- Teste 16: Após visualizar mensagem de erro, acioná-la através do botão de fechamento. Confirmar que o diálogo retorna ao estado inicial, permitindo nova escolha. Requisitos validados: DLG, GES;
- Teste 17: Percorrer sistematicamente todas as questões implementadas (identificação, confirmação de procedimento, etc.). Validar que cada justificativa apresentada está alinhada com diretrizes de segurança do paciente e protocolos de enfermagem. Requisitos validados: DLG, VPD;

Validação dos Efeitos Visuais

Estes testes validaram a reconstrução dos sistemas de partículas utilizando tecnologia Niagara.

- Teste 18: Posicionar as mãos na região da pia. Confirmar que o fluxo de água apresenta geometria cônica descendente, movimento contínuo e comportamento físico compatível com o projetado (considerando gravidade e velocidade terminal). Requisitos validados: AST, GES;
- Teste 19: Disparar o mecanismo do dispensador de sabão. Verificar emissão única com propriedades viscosas evidentes (menor velocidade, trajetória mais curta) sem reativação automática. Requisitos validados: AST, GES;

Validação da Apresentação do Paciente

Estes testes avaliaram a integração visual e animações do personagem virtual.

- Teste 20: Iniciar a aplicação. Confirmar que o paciente executa corretamente a transição animada entre os estados "Sitting" e "Sitted", finalizando com alinhamento adequado à cadeira. Requisitos validados: AST;

- Teste 21: Durante toda a duração da simulação, validar que o paciente permanece no estado "Sitted" sem transições indesejadas ou desalinhamentos visuais. Requisitos validados: AST;
- Teste 22: Movimentar-se pelo ambiente observando o paciente de múltiplas perspectivas. Verificar renderização consistente do modelo tridimensional independentemente do ângulo de observação. Requisitos validados: AST;

Validação Integrada dos Sistemas

Concluídos os testes modulares, realizou-se bateria de testes integrando múltiplos sistemas simultaneamente. Estes cenários complexos incluíram:

- Cenário 1: Executar sequência completa desde higienização manual até conclusão de diálogo com paciente, validando transições entre *Stories*, reposicionamento de marcadores e manutenção de estado do sistema.
- Cenário 2: Simular múltiplas tentativas incorretas em diferentes diálogos, verificando robustez do gerenciamento de estado e garantindo que progressão ocorre apenas mediante validação correta.
- Cenário 3: Reconfigurar Anchors através da interface de gerenciamento e re-executar procedimento completo, confirmando que alterações de posicionamento não comprometem lógica procedural nem integridade funcional.

Esta estratégia de validação exaustiva, combinando testes unitários, modulares e integrados, possibilitou identificação precoce de inconsistências e comportamentos inesperados. As correções foram aplicadas iterativamente, resultando em versão final estável e aderente aos requisitos estabelecidos para o sistema.

6 CONSIDERAÇÕES FINAIS

6.1 Conclusões do Projeto de Formatura

Este trabalho apresentou a modernização tecnológica do simulador VIDA Enfermagem, consolidando uma base robusta e atualizada para o desenvolvimento futuro de simulações educacionais em RV aplicadas ao treinamento do procedimento de PVP. A migração completa da Unreal Engine 4.19.2 para a versão 5.4.4 e a substituição do *hardware* obsoleto (*Oculus Rift* e *Leap Motion*) pelo Meta Quest 3 com rastreamento nativo de mãos não apenas resolveram problemas críticos de portabilidade e execução, mas também expandiram significativamente as possibilidades de interação natural e imersiva através da implementação de um sistema escalável de evolução de etapas do procedimento baseado em MEH.

A arquitetura modular hierárquica desenvolvida para o sistema de evolução de passos mostrou-se eficaz para modelar a natureza sequencial e procedural do treinamento médico, facilitando futuras expansões do sistema. O sistema de marcadores interativos com *feedback* visual em tempo real oferece aos estudantes uma forma intuitiva de progredir através das etapas, permitindo aprendizado com correção imediata de erros e validação pedagógica das ações executadas.

Os resultados dos testes de desempenho confirmaram que o protótipo modernizado atende aos padrões técnicos estabelecidos pela Meta para aplicações de RV, apresentando taxa média de quadros estável e utilização equilibrada de recursos computacionais. A unificação das versões fragmentadas entre INTERLAB e EERP-USP em um repositório centralizado com documentação técnica completa e controle de versionamento resolve definitivamente os problemas de reprodutibilidade que comprometiam o desenvolvimento colaborativo.

O desenvolvimento deste projeto evidenciou que a modernização de sistemas legados exige flexibilidade estratégica para redirecionar o escopo diante de obstáculos técnicos imprevistos. A colaboração interdisciplinar com a EERP-USP foi essencial para garantir que as decisões tecnológicas mantivessem alinhamento com as necessidades pedagógicas reais do treinamento em PVP, resultando em um simulador que respeita tanto os protocolos clínicos quanto os princípios de *design* de experiências imersivas.

Espera-se que este trabalho sirva como referência para futuras expansões do VIDA Enfermagem e inspire o desenvolvimento de novos simuladores educacionais em RV aplicados a diferentes procedimentos da área da saúde, contribuindo para a

formação de profissionais mais preparados, confiantes e competentes.

6.2 Contribuições

Este trabalho apresenta contribuições relevantes em duas frentes principais: técnica e educacional.

Na dimensão técnica, o projeto estabelece uma arquitetura modular baseada em MEH estruturada em níveis hierárquicos que pode ser adaptada para modelagem de diferentes procedimentos médicos sequenciais além da PVP. Adicionalmente, a documentação técnica produzida sobre os desafios enfrentados e superados ao longo do desenvolvimento deste trabalho constitui material de referência, escasso na literatura técnica, auxiliando tanto desenvolvedores que enfrentem desafios similares em projetos legados de RV quanto futuras equipes que assumam a continuidade do projeto VIDA, com a expectativa de reduzir o tempo necessário para compreensão do sistema e evitar retrabalho na solução de problemas já documentados.

Na dimensão educacional, este trabalho amplia o conjunto ainda limitado de pesquisas nacionais sobre aplicação de RV na educação em Enfermagem, contribuindo com evidências práticas sobre viabilidade técnica e aceitação por estudantes. A metodologia de validação pedagógica implementada, com *feedback* visual progressivo através de marcadores coloridos e mensagens contextualizadas de correção, apresenta abordagem replicável para outros treinamentos procedurais na área da saúde. A parceria consolidada entre INTERLAB e EERP-USP, nesse contexto, estabelece modelo de colaboração interdisciplinar que equilibra requisitos técnicos com necessidades pedagógicas reais, demonstrando como integrar conhecimento clínico especializado desde as fases iniciais de desenvolvimento para garantir fidelidade aos protocolos e relevância educacional das ferramentas criadas.

6.3 Perspectivas de Continuidade

A base tecnológica estabelecida por este trabalho abre múltiplas possibilidades de expansão do projeto VIDA Enfermagem. A implementação completa das etapas originalmente previstas do procedimento de PVP constitui continuidade natural e prioritária, aproveitando a arquitetura modular desenvolvida que facilita a adição de novas *stories* e *steps*. Esta expansão inclui a recuperação e integração de *assets* originais ainda não restaurados, como modelos tridimensionais de veias, equipamentos médicos e texturas anatômicas detalhadas que certamente enriquecerão ainda mais o realismo da simulação.

O sistema de colisões físicas representa outro aspecto que demanda aprimoramento em versões futuras. Atualmente, a simulação implementa detecção de colisão apenas entre o paciente e a cadeira, além de colisões entre os membros do próprio corpo do paciente, recursos que puderam ser reaproveitados da versão original do projeto. Diversos outros elementos da cena carecem de física de colisão adequada, resultando em interpenetrações visuais. As partículas de água e sabão atravessam as superfícies da pia e do dispensador, os objetos manipuláveis sobre a bancada podem afundar nela dependendo de como o usuário os posiciona, e os *widgets* de interface, exibidos sempre na altura do visor para garantir visibilidade, podem sobrepor-se visualmente ao paciente ou outros objetos tridimensionais. A implementação de um sistema robusto de colisões envolveria a configuração de *collision meshes* otimizadas para todos os objetos interativos e posicionamento inteligente de elementos de interface que considere a disposição espacial dos objetos da cena. Embora estas limitações não impeçam o funcionamento pedagógico do simulador, sua resolução contribuiria para a verossimilhança e imersão da experiência de RV.

Além disso, o modelo atual do paciente e suas animações podem ser substituídos por versões de maior qualidade visual e fidelidade anatômica, aproveitando bibliotecas mais recentes do Mixamo ou desenvolvendo modelos personalizados específicos para o contexto de Enfermagem. A integração de animações faciais e expressões corporais que demonstrem reações realistas do paciente durante o procedimento, como desconforto leve durante a punção e expressões de alívio após conclusão, aumentaria significativamente a imersão e o componente emocional do treinamento.

A Unreal Engine 5.4.4 oferece recursos avançados ainda não explorados neste trabalho que podem melhorar bastante a qualidade da simulação. O sistema Lumen de iluminação global dinâmica e reflexões em tempo real, por exemplo, poderia proporcionar ambientes mais realistas com menor custo computacional. A integração com o MetaHuman Creator permitiria a criação de pacientes virtuais fotorrealistas com variações de idade, gênero e características físicas, aumentando a diversidade de cenários de treinamento.

A estrutura de validação pedagógica implementada pode ser expandida para registrar métricas objetivas de desempenho do usuário, como tempo de execução de cada etapa, número de tentativas necessárias, erros cometidos e grau de assertividade. Estes dados poderiam gerar relatórios individualizados de progresso, identificar etapas que apresentam maior dificuldade e subsidiar intervenções pedagógicas direcionadas pelos docentes da EERP-USP.

O sistema de diálogo atual, implementado através de caixas textuais de sele-

ção, pode ser significativamente aprimorado através da integração com modelos de linguagem natural, permitindo que o paciente virtual responda de forma adaptativa às perguntas do estudante e simule diferentes perfis de paciente, como ansioso, colaborativo ou com dificuldades de comunicação. A captura e processamento de áudio através dos microfones integrados do Meta Quest 3 viabilizaria interações por voz, tornando a experiência mais realista. A própria Unreal Engine 5 oferece ferramentas nativas de integração com serviços de IA conversacional que poderiam ser exploradas para implementar esta funcionalidade de forma otimizada.

REFERÊNCIAS

ADOBE. *Mixamo*. 2025. Plataforma de personagens e animações 3D. Disponível em: <<https://www.mixamo.com/>>. Acesso em: 5 dez. 2025. Citado nas páginas 64 e 67.

ALMEIDA, D. P. de. Nível de conhecimento e desempenho dos acadêmicos de enfermagem quanto à técnica correta da punção venosa periférica. *Enfermagem Brasil*, v. 11, n. 1, p. 10–19, 2012. Citado na página 21.

BRASIL. Agência Nacional de Vigilância Sanitária. *RDC nº 36/2013 – Institui ações para a segurança do paciente em serviços de saúde*. Brasília, 2013. Disponível em: <https://bvsms.saude.gov.br/bvs/saudelegis/anvisa/2013/rdc0036_25_07_2013.html>. Acesso em: 23 fev. 2025. Citado na página 63.

BRASIL. Agência Nacional de Vigilância Sanitária. *Medidas de prevenção de infecção relacionada à assistência à saúde*. Brasília, 2017. 49–65 p. Disponível em: <<https://www.gov.br/anvisa/pt-br>>. Acesso em: 23 fev. 2025. Citado nas páginas 91, 93 e 95.

CHEN, F.-Q. et al. Effectiveness of virtual reality in nursing education: meta-analysis. *Journal of Medical Internet Research*, v. 22, n. 9, p. e18290, 2020. Citado na página 22.

COMMONS, W. *Meta Quest 3 display unit*. 2023. Disponível em: <https://en.wikipedia.org/wiki/Meta_Quest_3#/media/File:Meta_Quest_3_display_unit.jpg>. Acesso em: 18 mar. 2025. Citado na página 41.

Conselho Federal de Enfermagem. *Código de Ética dos Profissionais de Enfermagem*. Brasília, 2017. Disponível em: <http://www.cofen.gov.br/resolucao-cofen-no-5642017_59145.html>. Acesso em: 23 fev. 2025. Citado na página 63.

ENGINE, U. *Unreal Editor for Fortnite | Build Games and Experiences*. Disponível em: <<https://www.unrealengine.com/en-US/uses/uefn-unreal-editor-for-fortnite>>. Acesso em: 17 mar. 2025. Citado na página 40.

ENGINE, U. *Batman: Arkham City*. 2012. Disponível em: <<https://www.unrealengine.com/pt-BR/blog/batman-arkham-city>>. Acesso em: 17 mar. 2025. Citado na página 40.

Escola de Enfermagem de Ribeirão Preto – USP. *Roteiro de ensino da intervenção "Coleta de sangue venoso a vácuo"*. Ribeirão Preto, 2024. Disciplinas ERG 0238 e ERG 0346 – Fundamentos de Enfermagem. Citado nas páginas 88, 91, 93, 95, 96 e 99.

GAMES, E. *Unreal Engine 4.19 Released*. 2018. Disponível em: <<https://www.unrealengine.com/en-US/blog/unreal-engine-4-19-released>>. Citado na página 46.

GAMES, E. *Unreal Engine 5: Next-Generation Real-Time 3D Creation Tool*. 2022. Disponível em: <<https://www.unrealengine.com/en-US/unreal-engine-5>>. Citado na página 46.

- GAMES, E. *Blueprints Visual Scripting in Unreal Engine*. [S.l.], 2025. Disponível em: <<https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine>>. Acesso em: 16 mar. 2025. Citado na página 40.
- GAUDIOSI, J. *Mortal Kombat X Fights to the Finish*. 2015. Disponível em: <<https://www.unrealengine.com/fr/blog/mortal-kombat-x-fights-to-the-finish>>. Acesso em: 17 mar. 2025. Citado na página 40.
- GAUDIOSI, J. *How Unreal Engine 4 Provided Street Fighter V's Knockout Punch*. 2016. Disponível em: <<https://www.unrealengine.com/pt-BR/developer-interviews/how-unreal-engine-4-provided-street-fighter-vs-knockout-punch>>. Acesso em: 17 mar. 2025. Citado na página 40.
- MAKRINI, I. E. et al. A hierarchical finite-state machine-based task allocation framework for human-robot collaborative assembly tasks. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. [S.l.: s.n.], 2022. p. 10238–10244. Citado na página 54.
- META. *Unreal HandGameplay Sample – Oculus Samples*. 2024. Disponível em: <<https://github.com/oculus-samples/Unreal-HandGameplay>>. Acesso em: 26 jun. 2025. Citado nas páginas 50 e 52.
- META. *Meta Quest 3 - Especificações*. [S.l.], 2025. Disponível em: <<https://www.meta.com/quest/quest-3/#specs>>. Acesso em: 16 mar. 2025. Citado na página 41.
- META. *Meta Quest Performance Optimization Guide*. [S.l.], 2025. Disponível em: <<https://developers.meta.com/horizon/resources/vrc-quest-performance-1/>>. Acesso em: 17 dez. 2025. Citado na página 68.
- META. *Meta XR Interaction SDK for Unreal – Version 71.0.0*. [S.l.], 2025. Disponível em: <<https://developers.meta.com/horizon/downloads/package/meta-xr-interaction-sdk-unreal/71.0.0>>. Acesso em: 12 ago. 2025. Citado nas páginas 37, 46, 52 e 61.
- META. *MQDH Logs Metrics*. [S.l.], 2025. Disponível em: <<https://developers.meta.com/horizon/documentation/native/android/ts-mqdh-logs-metrics/>>. Acesso em: 17 dez. 2025. Citado na página 68.
- META. *OVR Best Practices for Unity*. [S.l.], 2025. Disponível em: <<https://developers.meta.com/horizon/documentation/unity/ts-ovr-best-practices>>. Acesso em: 17 dez. 2025. Citado na página 69.
- META. *OVRStats for Unity*. [S.l.], 2025. Disponível em: <<https://developers.meta.com/horizon/documentation/unity/ts-ovrstats>>. Acesso em: 17 dez. 2025. Citado nas páginas 68 e 69.
- MILGRAM, P. et al. Augmented reality: a class of displays on the reality-virtuality continuum. 1995. Citado na página 26.
- Ministério da Saúde. *Protocolo de Identificação do Paciente*. Brasília, 2013. Anvisa, RDC nº 36/2013 – Segurança do Paciente. Disponível em: <<https://www.gov.br/saude/pt-br>>. Acesso em: 23 fev. 2025. Citado na página 63.

NEDACHI, N. D. L. *Efetividade de uma equipe para punção venosa em pacientes com câncer: um estudo retrospectivo*. Tese (Doutorado) — Universidade de São Paulo, 2025. Citado na página 21.

OBERG, R.; PROBASCO, L.; ERICSSON, M. *Applying Requirements Management with Use Cases*. [S.l.], 2001. Citado na página 34.

PARK, D. *How to create spatial interactions with Meta XR Interaction SDK*. 2024. Medium. Disponível em: <<https://dongyoonpark.medium.com/how-to-create-spatial-interactions-with-meta-xr-interaction-sdk-3922a837ea0d>>. Acesso em: 9 dez. 2025. Citado na página 53.

Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK® Guide) – Seventh Edition and The Standard for Project Management*. 7th. ed. Newtown Square, PA: Project Management Institute, 2021. ISBN 978-1-62825-664-2. Citado nas páginas 46 e 47.

ROSEN, K. H. *Discrete Mathematics and Its Applications*. 8th. ed. New York, NY: McGraw-Hill Education, 2019. ISBN 978-1259676512. Citado na página 28.

SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL. *Recomendações da Sociedade Brasileira de Patologia Clínica/Medicina Laboratorial para coleta de sangue venoso*. Barueri, SP, 2010. Disponível em: <<http://bibliotecasbpc.org.br/pags/view.archive.php?ID=173&PATH=pdf>>. Acesso em: 23 fev. 2025. Citado nas páginas 88, 91, 93 e 95.

SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL. *Recomendações da Sociedade Brasileira de Patologia Clínica/Medicina Laboratorial (SBPC/ML): coleta e preparo da amostra biológica*. Barueri, SP, 2014. Disponível em: <http://www.sbpc.org.br/upload/conteudo/livro_coleta_biologica2013.pdf>. Acesso em: 23 fev. 2025. Citado nas páginas 88, 91, 93 e 95.

SOUSA, L. C. A. de et al. Realidade virtual no ensino do procedimento de punção venosa periférica: Revisão sistemática virtual reality in teaching peripheral venipuncture procedure: Systematic review. *Brazilian Journal of Development*, v. 7, n. 12, p. 113235–113247, 2021. Citado nas páginas 22, 23 e 25.

SOUZA-JÚNIOR, V. D. *Simulação de realidade virtual imersiva no procedimento de punção venosa periférica para coleta de sangue a vácuo*. Tese (Tese (Doutorado em Enfermagem Fundamental)) — Escola de Enfermagem de Ribeirão Preto, University of São Paulo, 2018. Acesso em: 22 fev. 2025. Citado nas páginas 23, 33 e 47.

SOUZA-JÚNIOR, V. D. et al. Vida-nursing v1.0: immersive virtual reality in vacuum blood collection among adults. *Revista Latino-Americana de Enfermagem*, v. 28, p. e3263, 2020. Citado nas páginas 24, 27, 28 e 64.

TORI, R.; HOUNSELL, M. d. S. (Ed.). *Introdução a Realidade Virtual e Aumentada*. Porto Alegre: Editora SBC, 2018. Citado na página 26.

TORRES, M. M. *Punção venosa periférica: avaliação do desempenho dos profissionais de enfermagem de um hospital geral do interior paulista*. Tese (Doutorado) — Universidade de São Paulo, 2003. Citado na página 21.

TORRES, M. M.; ANDRADE, D. d.; SANTOS, C. B. d. Punção venosa periférica: avaliação de desempenho dos profissionais de enfermagem. *Revista Latino-Americana de Enfermagem*, SciELO Brasil, v. 13, p. 299–304, 2005. Citado nas páginas 21 e 22.

YANNAKAKIS, M. Hierarchical state machines. In: LEEUWEN, J. van et al. (Ed.). *Theoretical Computer Science: Exploring New Frontiers of Theoretical Informatics*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2000. p. 315–330. ISBN 978-3-540-44929-4. Citado nas páginas 29 e 30.

APÊNDICES

APÊNDICE A - DESAFIOS TÉCNICOS E LIÇÕES APRENDIDAS

A.1 Processo Iterativo de Migração de Versão

Durante a migração do projeto da versão 4.19 para a 5.4.4 da Unreal Engine, diversas abordagens foram testadas antes da solução final. Inicialmente, tentou-se a migração direta para a versão 5.3.2, que não obteve sucesso devido a erros de compilação persistentes. As mensagens de erro indicavam problemas na cópia automática de arquivos, mas as causas específicas não eram claras.

Subsequentemente, testou-se a migração direta para a versão 5.4.4, que apresentou os mesmos problemas. Tentativas de recompilação manual do arquivo *.sln* no Visual Studio também não foram suficientes para resolver as incompatibilidades.

Lição aprendida: Para migrações entre versões muito distantes da Unreal Engine (4.X para 5.X), o recurso nativo de migração pode não ser confiável. Recomenda-se considerar desde o início a transferência manual de *assets* como estratégia principal, ou realizar migrações incrementais passando por versões intermediárias.

A.2 Estratégias de Instanciação Avaliadas para o Sistema de Evolução

A definição da abordagem de instanciação dinâmica do sistema de evolução resultou de um processo iterativo de avaliação de diferentes estratégias, considerando a complexidade do fluxo contínuo de etapas e sub-etapas do procedimento de punção venosa periférica. O objetivo era delinear uma solução que permitisse a instanciação de ambas as classes sem prejudicar a progressão do procedimento e a experiência de imersão do usuário, além de não afetar negativamente o desempenho da simulação.

Primeira estratégia - Referências fortes: Utilizou-se o uso de referências fortes a objetos, instanciando todos os *steps* e *stories* de uma só vez na cena. Nessa abordagem, todas as instâncias dos *assets* referenciados pelos objetos precisariam estar carregadas na memória, aumentando o tempo inicial de carregamento. Utilizou-se um diretório central para unificar todas as instâncias necessárias ao procedimento. Contudo, devido à baixa escalabilidade no contexto do procedimento de punção, essa solução foi descartada.

Segunda estratégia - Referências fracas com carregamento assíncrono:

Utilizava referências fracas a objetos para implementar um carregamento assíncrono das instâncias definidas, mantendo apenas o *storyteller* na cena. Nessa solução, armazenava-se apenas o caminho aos *assets* referenciados na memória, carregando o *asset* efetivamente apenas quando o *storyteller* solicitasse uma *story* via carregamento assíncrono. Essa solução também foi descartada, pois uma abordagem mais modular e simples foi concebida.

Solução final - Referências a classes: Utilizam-se referências a classes ao invés de referências a objetos instanciados, permitindo que os componentes sejam criados dinamicamente conforme necessário. Essa abordagem combina as vantagens das soluções anteriores: baixo consumo de memória (apenas elementos ativos instanciados) e alta modularidade (novas *stories* e *steps* adicionados via subclasses).

Lição aprendida: Em sistemas hierárquicos complexos com múltiplos níveis de abstração, referências a classes com instanciação dinâmica oferecem melhor equilíbrio entre performance, modularidade e simplicidade de desenvolvimento do que pré-instanciação completa ou carregamento assíncrono de objetos.

APÊNDICE B - RESULTADOS DOS TESTES**B.1 Protótipo inicial (n=1)**

Métrica	Média
<i>available memory</i>	3124.57 (MB)
<i>app gpu time</i>	9643.63 (μ s)
<i>gpu utilization</i>	91.21 (%)
<i>used memory (app pss)</i>	571.82 (MB)
<i>foveation level</i>	0.00
<i>battery level</i>	81.04 (%)
<i>cpu level</i>	3.12
<i>average frame rate</i>	71.75 (FPS)
<i>early frame count</i>	3.64
<i>eye buffer width</i>	1622.00 (px)
<i>gpu level</i>	4.00
<i>stale frame count</i>	1.35
<i>eye buffer height</i>	1760.00 (px)
<i>cpu utilization</i>	72.71 (%)

APÊNDICE C - ESTRUTURA DE VALIDAÇÃO PEDAGÓGICA DAS ETAPAS DE INTERAÇÃO

Este apêndice apresenta o mapeamento completo das interações de diálogo desenvolvidas em colaboração com a EERP-USP. A estrutura foi baseada nos roteiros de ensino das disciplinas ERG 0238 e ERG 0346 - Fundamentos de Enfermagem, seguindo protocolos nacionais e internacionais de segurança do paciente e boas práticas de coleta de sangue venoso.

Cada tabela apresenta uma etapa completa do procedimento, incluindo contexto, opções de resposta, justificativas pedagógicas, importância clínica, riscos associados e referências normativas. Esta documentação serve como base para o desenvolvimento de novas etapas e para a validação do conteúdo pedagógico implementado no simulador.

C.1 Investigação de Histórico de Alergias

Tabela 6.2: Estrutura de validação pedagógica para investigação de alergias.

Campo	Conteúdo
Etapa	Preparação (Item 3).
Contexto	Identificar se o paciente sabe se possui alguma alergia.
Pergunta	Escolha a forma mais adequada de investigar alergias do paciente:
Opções de Resposta	A) “O Sr. sabe se possui alergia a alguma solução de passar na pele (álcool, clorexidina ou iodo) ou ao látex da luva?” (<i>Correta</i>)
	B) “O Sr. tem alguma alergia?” (<i>Incorreta</i>)
	C) “Vou usar álcool 70%. O Sr. tem alergia?” (<i>Incorreta</i>)
	D) “Podemos prosseguir com o procedimento padrão?” (<i>Incorreta</i>)

Continua na próxima página

Tabela 6.2 – Continuação

Campo	Conteúdo
Justificativa (A)	Esta pergunta abrange todos os materiais potencialmente alergênicos que serão utilizados no procedimento: antissépticos (álcool, clorexidina, iodo) e látex (luvas e torniquete). Permite ao profissional selecionar alternativas seguras antes de iniciar.
Justificativa (B)	Pergunta muito genérica que não direciona o paciente para as substâncias específicas que serão utilizadas no procedimento. O paciente pode esquecer de mencionar alergias relevantes ou relatar alergias não pertinentes ao contexto.
Justificativa (C)	Investiga apenas uma das substâncias potencialmente alergênicas, omitindo látex (luvas e torniquete) e outras opções de antissépticos. Além disso, já pressupõe o uso de álcool antes de verificar possível alergia.
Justificativa (D)	Não investiga especificamente alergias e pressupõe que o procedimento padrão é adequado. Esta abordagem não permite identificar necessidade de adaptações nos materiais.
Importância	Para que o paciente não seja exposto a um alérgeno conhecido.
Risco Associado	O paciente apresentar uma reação alérgica na(s) áreas expostas ao antisséptico, ou as luvas e/ou ao torniquete, durante ou após a coleta do sangue.
Referência	SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2010); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2014); Escola de Enfermagem de Ribeirão Preto – USP (2024).

Continua na próxima página

Tabela 6.2 – Continuação

Campo	Conteúdo
Observações Adicionais	Nenhuma.

C.2 Explicação do Procedimento

Tabela 6.3: Estrutura de validação pedagógica para explicação do procedimento.

Campo	Conteúdo
Etapas	Preparação (Item 4).
Contexto	Averiguar a experiência prévia do paciente, explicar o que será feito e identificar a capacidade de cooperação do paciente.
Pergunta	“O Sr. já fez alguma coleta de sangue a vácuo?” Como proceder após a resposta do paciente?
Opções de Resposta	A) Se sim, prosseguir diretamente para o próximo item. (<i>Correta</i>) B) Se não, explicar detalhadamente o procedimento antes de prosseguir. (<i>Correta</i>) C) Independente da resposta, prosseguir sem explicações adicionais. (<i>Incorreta</i>)
Justificativa (A)	Quando o paciente conhece o procedimento sofre menos ansiedade (que pode alterar parâmetros fisiológicos como cortisol, glicose e predispor a síncope). Estudante pode prosseguir para o próximo item.
Justificativa (B)	A informação reduz ansiedade (que pode alterar parâmetros fisiológicos como cortisol, glicose e predispor a síncope). Quando o paciente desconhece o procedimento, o estudante explica o que será feito antes de prosseguir: local da punção, sensação esperada, duração aproximada e importância da imobilidade.

Continua na próxima página

Tabela 6.3 – Continuação

Campo	Conteúdo
Justificativa (C)	Não avaliar a experiência prévia do paciente e não fornecer explicações adequadas aumenta a ansiedade e pode comprometer a cooperação durante o procedimento, além de ferir princípios éticos de autonomia e consentimento informado.
Importância	Essa interação é necessária para que o estudante identifique o quanto e como o paciente pode colaborar na realização do procedimento.
Risco Associado	Risco de perda da amostra coletada devido a alteração de parâmetros, além de síncope e reatividade do paciente.
Referência	BRASIL. Agência Nacional de Vigilância Sanitária (2017); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2010); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2014); Escola de Enfermagem de Ribeirão Preto – USP (2024).
Observações Adicionais	Nenhuma.

C.3 Obtenção do Consentimento

Tabela 6.4: Estrutura de validação pedagógica para obtenção do consentimento.

Campo	Conteúdo
Etapas	Preparação (Item 5).
Contexto	Identificar se o paciente permite que a(s) amostra(s) de sangue sejam colhidas.
Pergunta	“O Sr. autoriza que eu colete o sangue necessário para os seus exames?” Como proceder após a resposta?
Opções de Resposta	A) Se sim, prosseguir para o próximo item. (<i>Correta</i>) B) Se não, interromper e esclarecer dúvidas do paciente. (<i>Correta</i>) C) Realizar o procedimento independente da resposta, pois há solicitação médica. (<i>Incorreta</i>)
Justificativa (A)	É requisito ético para procedimentos. Comunicação clara segue recomendações de boas práticas para segurança do paciente. O consentimento verbal explícito permite que o estudante prossiga para o próximo item.
Justificativa (B)	É requisito ético para procedimentos. Se o paciente recusar, o estudante deve interromper, investigar os motivos da recusa, esclarecer dúvidas, reforçar a importância do exame e, se necessário, comunicar a equipe. O paciente tem direito de recusar procedimentos.

Continua na próxima página

Tabela 6.4 – Continuação

Campo	Conteúdo
Justificativa (C)	Realizar procedimentos sem consentimento do paciente consciente e capaz constitui violação ética e legal do princípio da autonomia, mesmo havendo solicitação médica. O consentimento é requisito fundamental que não pode ser ignorado.
Importância	Essa interação é necessária para que o estudante cumpra com os requisitos éticos para procedimentos, além de ser uma recomendação de boa prática para segurança do paciente, e também do profissional.
Risco Associado	Risco de dano físico e emocional ao paciente.
Referência	BRASIL. Agência Nacional de Vigilância Sanitária (2017); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2010); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2014); Escola de Enfermagem de Ribeirão Preto – USP (2024).
Observações Adicionais	Nenhuma.

C.4 Verificação das Condições Pré-analíticas

Tabela 6.5: Estrutura de validação pedagógica para verificação de condições pré-analíticas.

Campo	Conteúdo
Etapa	Preparação (Item 6)
Contexto	Identificar se o paciente se preparou para a coleta do(s) exame(s).
Pergunta	Selecione uma das perguntas que devem ser feitas para verificar as condições pré-analíticas? (Há mais de uma correta)
Opções de Resposta	A) “Qual o horário em que o Sr. realizou a última refeição?” (<i>Correta</i>)
	B) “Há quanto tempo o Sr. está sentado/em repouso?” (<i>Correta</i>)
	C) “O Sr. está usando medicamentos atualmente?” (<i>Correta</i>)
	D) Não é necessário verificar essas condições, basta coletar conforme solicitação (<i>Incorreta</i>)
Justificativa (A)	De acordo com os exames solicitados, o estudante avalia se o tempo de jejum está adequado. Diferentes exames requerem diferentes períodos de jejum (8-12 horas para perfil lipídico, 8 horas para glicemia de jejum). Se inadequado, reagendar a coleta.
Justificativa (B)	De acordo com os exames solicitados, o estudante avalia se o tempo de repouso está adequado (geralmente 15 minutos). Exercício físico recente pode alterar diversos parâmetros laboratoriais. Se inadequado, aguardar ou reagendar.

Continua na próxima página

Tabela 6.5 – Continuação

Campo	Conteúdo
Justificativa (C)	O estudante deve registrar os medicamentos em uso antes de prosseguir. Diversos medicamentos interferem em resultados laboratoriais. O registro permite interpretação adequada dos resultados pelo médico solicitante.
Justificativa (D)	Não verificar as condições pré-analíticas compromete a qualidade e confiabilidade dos resultados laboratoriais, podendo levar a diagnósticos incorretos e condutas terapêuticas inadequadas. A fase pré-analítica é responsável por 60-70% dos erros laboratoriais.
Importância	Variáveis pré-analíticas (jejum, posição, exercícios, medicamentos) alteram resultados laboratoriais. De acordo com o analito a ser dosado há diretrizes específicas para cada exame. Registrar essas condições é essencial para interpretação adequada dos resultados.
Risco Associado	Risco de falso positivo ou negativo, e interpretação inadequada do resultado.
Referência	BRASIL. Agência Nacional de Vigilância Sanitária (2017); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2010); SOCIEDADE BRASILEIRA DE PATOLOGIA CLÍNICA/MEDICINA LABORATORIAL (2014); Escola de Enfermagem de Ribeirão Preto – USP (2024).
Observações Adicionais	Nenhuma.

APÊNDICE D - DESCRIÇÃO COMPLETA DO PROCEDIMENTO DE COLETA DE SANGUE VENOSO A VÁCUO

Este apêndice apresenta parte do roteiro do procedimento de coleta de sangue venoso a vácuo, conforme roteiro de ensino das disciplinas ERG 0238 e ERG 0346 - Fundamentos de Enfermagem da EERP-USP (Escola de Enfermagem de Ribeirão Preto – USP, 2024). A descrição completa serve como referência para compreensão das etapas implementadas no simulador e contextualiza as interações pedagógicas descritas no Apêndice 6.3.

D.1 Classificação de Criticidade das Etapas

As etapas do procedimento são classificadas conforme seu impacto na segurança do paciente e/ou do profissional em três níveis:

- **C** = Crítico
- **MC** = Muito Crítico
- **EC** = Extremamente Crítico

D.2 Descrição Completa do Procedimento

Fase de Preparação

1. Higienizar as mãos; **(EC)**
2. Identificar o paciente: perguntar nome completo e data de nascimento, ler pulseira de identificação e confrontar com identificação do leito e/ou do pedido/etiquetas do(s) exame(s); **(EC)**
3. Investigar histórico de alergias; **(EC)**
4. Explicar o procedimento; **(C)**
5. Obter o consentimento do paciente para a coleta da amostra sanguínea; **(C)**
6. Certificar-se que o paciente cumpriu as recomendações conforme o(s) exame(s) a serem coletados (jejum, repouso, uso de medicamentos, entre outras); **(EC)**
7. Garantir a privacidade do paciente e boa iluminação local; **(C)**
8. Posicionar o braço do paciente estendido sobre uma superfície estável; **(C)**
9. Expor o braço do paciente; **(C)**

10. Colocar e apertar o torniquete no braço, com as pontas voltadas para cima; **(EC)**
11. Inspeccionar e palpar a rede venosa do braço quanto à profundidade, elasticidade, trajeto e calibre; **(EC)**
12. Selecionar a veia que será puncionada; **(MC)**
13. Remover o torniquete; **(EC)**
14. Repetir os itens 8 a 13 no outro braço, caso necessário; **(C)**
15. Higienizar as mãos; **(EC)**
16. Reunir o material na bandeja; **(MC)**

Fase de Execução

17. Higienizar as mãos; **(EC)**
18. Abrir a agulha/escalpe e conectar a agulha revestida de borracha no adaptador; **(EC)**
19. Posicionar o braço do paciente estendido sobre uma superfície estável; **(MC)**
20. Colocar o torniquete sob o braço do paciente, de 10 a 15 cm acima do local a ser puncionado; **(C)**
21. Inspeccionar e palpar a veia selecionada, se necessário; **(C)**
22. Realizar a antisepsia da pele do local selecionado utilizando lenço esterilizado umedecido com álcool a 70% ou algodão umedecido com álcool a 70% em movimento espiral, centrífugo, firme e único; **(EC)**
23. Colocar os EPIs: máscara, óculos de proteção individual e luvas; **(EC)**
24. Aguardar a pele secar espontaneamente por completo; **(MC)**
25. Apertar o torniquete no braço, com as pontas voltadas para cima; **(EC)**
26. Remover a capa protetora da agulha/escalpe; **(C)**
27. Estirar a pele, com a mão não dominante; **(MC)**
28. Segurar o adaptador ou o escalpe pelas asas, com a mão dominante; **(MC)**
29. Posicionar a agulha em um ângulo de 10° a 30° (compatível com a profundidade da veia) com o bisel voltado para cima; **(EC)**
30. Comunicar o paciente que a agulha/escalpe será introduzida(o); **(C)**
31. Introduzir a agulha/escalpe: **(EC)**
 - Agulha: introduzir o bisel até alcançar o lúmen da veia e não a transfixar;

- Escalpe: introduzir até observar retorno sanguíneo pela extensão;
32. Soltar a pele; **(C)**
 33. Introduzir o tubo a vácuo no adaptador e empurrá-lo em direção à agulha revestida de borracha até perfurar o diafragma da tampa, com a mão não dominante; **(EC)**
 34. Observar o fluxo de sangue para dentro do tubo; **(MC)**
 35. Aguardar o enchimento do tubo; **(EC)**
 36. Afrouxar ou remover o torniquete. Se houver mais de um tubo de coleta, repetir os passos 38 a 40; **(EC)**
 37. Retirar o tubo do adaptador; **(MC)**
 38. Homogeneizar os tubos de sangue, realizando movimentos suaves de cinco a dez vezes; **(EC)**
 39. Colocar o algodão seco próximo ao local puncionado para apoiar a pele; **(MC)**
 40. Remover a agulha/escalpe no mesmo ângulo em que foi introduzida(o); **(MC)**
 41. Comprimir o local da punção com o algodão seco; **(MC)**
 42. Direcionar a agulha/escalpe para baixo e acionar o dispositivo de segurança; **(EC)**
 43. Colocar o conjunto agulha-adaptador/escalpe na bandeja; **(MC)**
 44. Manter a compressão, ou solicitar ao paciente que comprima, o local da punção por 1 a 2 minutos, elevar o braço e mantê-lo estendido; **(MC)**
 45. Retirar o algodão e avaliar a necessidade de aplicar o curativo adesivo; **(C)**
 46. Identificar o(s) tubo(s) sem ocluir a área de visualização do sangue; **(EC)**
 47. Mostrar o(s) tubo(s) para o paciente e pedir que confirme seus dados na(s) etiqueta(s); **(C)**

Fase de Conclusão

48. Posicionar o paciente confortavelmente; **(C)**
49. Retirar a luva da mão não dominante, e com ela retirar os óculos de proteção individual e a máscara; **(C)**
50. Recompor a unidade; **(C)**
51. Acondicionar os tubos em local definido no protocolo institucional; **(MC)**

- 52. Desprezar os perfurocortantes em recipiente rígido e os demais materiais em local adequado, com a mão dominante; **(EC)**
- 53. Retirar a luva da mão dominante e desprezá-la; **(C)**
- 54. Desinfetar os materiais não descartáveis, conforme protocolo institucional; **(EC)**
- 55. Higienizar as mãos; **(EC)**
- 56. Registrar a intervenção no prontuário; **(C)**

Fase de Avaliação

- 57. Investigar a presença de dor aguda, dormência ou hematoma no local. **(C)**

D.3 Etapas Implementadas no Simulador

O simulador desenvolvido implementa as seis primeiras etapas da fase de preparação, classificadas como críticas (C) ou extremamente críticas (EC), conforme detalhado no Apêndice D:

Tabela 6.6: Correspondência entre etapas do roteiro e implementação no simulador.

Item Roteiro	Etapas	Criticidade	Apêndice D
1	Higienizar as mãos	EC	-
2	Identificar o paciente	EC	-
3	Investigar histórico de alergias	EC	D.1
4	Explicar o procedimento	C	D.2
5	Obter consentimento do paciente	C	D.3
6	Certificar cumprimento de recomendações	EC	D.4

Fonte: Elaborado pelos autores com base em Escola de Enfermagem de Ribeirão Preto – USP (2024).

A seleção dessas etapas justifica-se por sua alta criticidade para a segurança do paciente, por serem fundamentais para o estabelecimento da relação profissional-paciente e por representarem o conjunto de verificações pré-procedimento essenciais que devem ser realizadas antes de qualquer intervenção invasiva.